

JAVA SDE-2 INTERVIEW PREPARATION SERIES

SYSTEM DESIGN - BOOK 05 OF 05 - STUDY STEP SD-05

High-Level Design Case Book

Worked System Design Cases with the Arithmetic That Chooses the Architecture

BY

Vinay Reddy Kalluri

Work the design prompts interviewers actually use - URL shortener, distributed cache, news feed, chat, typeahead, ride matching, payments, metrics and object storage - with a fixed method and the numbers that decide each architecture.

ESTIMATION | KEYS | PARTITIONING | FAN-OUT | ORDERING | GEOHASH | EXACTLY-ONCE | DURABILITY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [SD 04 - Low-Level Design and Machine Coding](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This book applies the building blocks from SD-02 to the high-level design prompts used in SDE-2 loops. Every case follows one 45-minute method, and every architectural choice is justified by arithmetic that an executable Java 21 model recomputes and checks.

Readiness map

Decision	Evidence
START HERE	The prompt says design a product or service at scale, not implement a class model; The interviewer expects capacity numbers before technology choices; The round has 45 to 60 minutes and ends with failure and scale follow-ups; The difficulty is allocation, skew, ordering, freshness, spatial data, or volume.
PREPARE FIRST	SD-02 Distributed Systems and System Design, or equivalent knowledge of partitioning, replication, and queues; SD-04 is helpful for the component-level detail behind several cases; Comfort with back-of-the-envelope arithmetic.
MOVE ON WHEN	Run a fixed 45-minute design method with written requirements and non-goals; Produce an estimate in under five minutes and let it choose the architecture; Explain key generation, consistent hashing, hybrid fan-out, per-conversation ordering, geohash search, and exactly-once payments with numbers; Compare durability schemes and storage tiers by computed loss probability and bytes; Map an unfamiliar prompt to its dominant difficulty and answer the standard follow-ups.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

System Design Segment Position

SD 04 - Low-Level Design and Machine Coding	YOU ARE HERE SD 05 - High-Level Design Case Book	SEGMENT COMPLETE
---	--	------------------

A note from Vinay

The diagram is the easy part of a design round. Write the requirements down, do the arithmetic before you name a technology, and spend your deep dive where the hardest requirement meets the load. When a number surprises you, say so out loud - that is usually the moment the interviewer starts to trust your design.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - The HLD Case Method and Estimation Toolkit	4
Chapter 2 - URL Shortener: Key Generation and the Read Path	10
Chapter 3 - Distributed Cache and Key-Value Store	15
Chapter 4 - News Feed: Fan-Out and the Celebrity Problem	20
Chapter 5 - Chat and Presence: Ordering, Delivery, and Connections	25
Chapter 6 - Typeahead and Search: Tries, Top-K, and Bloom Filters	30
Chapter 7 - Proximity and Ride Matching: Geohash and Moving Points	35
Chapter 8 - Payments and Ticket Booking: Exactly-Once Effects	40
Chapter 9 - Metrics Pipeline and Object Storage: Volume and Durability	45
Chapter 10 - Case Bank, Follow-Ups, and Scoring	50
Chapter 11 - High-Level Design Case Drills	55
Chapter 12 - High-Level Design Case Drills: Worked Solutions	57
Chapter 13 - Executable High-Level Design Model	62
Part II - Practice and Handoff	81

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - The HLD Case Method and Estimation Toolkit

Learning objectives

By the end of this chapter, you should be able to:

- run a 45-minute high-level design round with a fixed structure and a time budget;
- turn a vague prompt into functional requirements, non-functional requirements, and explicit non-goals;
- produce a capacity estimate in under five minutes, with rounding that is honest about its error;
- choose the one or two components that deserve a deep dive, and defend the choice; and
- recognise the difference between an SDE-1 and an SDE-2 answer to the same prompt.

WHY IT WORKS | Why this matters at SDE-2

SD-02 taught the building blocks: partitioning, replication, consistency, queues, and failure handling. This book applies them to the prompts that interviewers actually use. Every case follows the same template, because the template is the skill. Candidates rarely fail a design round for not knowing what a message queue is. They fail because they draw twelve boxes in the first ten minutes, never state a number, and spend the deep dive on the component the interviewer cared least about.

At SDE-2 the bar moves in three specific ways. You are expected to **drive** the conversation rather than wait for questions; to **quantify** before choosing technology; and to **name the failure mode** of each component you add. An SDE-1 answer to "design a URL shortener" is a correct diagram. An SDE-2 answer is the same diagram plus "at 3,900 reads per second a single cache node is enough, the interesting problem is key generation, and here is why random 7-character keys will collide five million times over five years".

WHY IT WORKS | First-principles model

A high-level design is **a set of data flows sized against a load, with a stated failure behaviour for each flow.**

- **Flows** come from the functional requirements: create a short link, follow a short link, post, read a feed.
- **Load** comes from the estimate: requests per second on each flow, bytes stored, bytes moved.
- **Failure behaviour** comes from the non-functional requirements: what may be stale, what may be lost, what must never be duplicated.

Everything on the whiteboard should trace back to one of those three. A box that serves no flow, handles no load, and changes no failure behaviour is decoration.

Specification boundary: there is no standard answer to an HLD prompt, and interviewers know it. What they grade is whether your choices follow from your stated requirements and numbers. Two candidates can choose different databases and both pass; a candidate who cannot say *why* their database fits the access pattern will not.

Core terminology

- **Functional requirement:** a behaviour the system must provide ("follow a short link").
- **Non-functional requirement:** a quality it must have ("p99 redirect under 50 ms", "no link ever points to the wrong URL").
- **Non-goal:** something explicitly out of scope, stated so that it is not silently assumed.
- **Back-of-the-envelope estimate:** a capacity calculation accurate to within a factor of about two.
- **Hot path:** the flow that carries most of the traffic or is most latency-sensitive.
- **Deep dive:** the part of the round where one component is designed in detail.
- **Read/write ratio:** the proportion of read to write traffic, which drives caching and replication decisions.

Detailed mechanics

The 45-minute structure

TEXT		
minutes	step	artifact on the board
0-5	requirements	3-5 functional, 3-5 non-functional, 2-3 non-goals
5-10	estimate	QPS per flow, storage, bandwidth, read/write ratio
10-15	API and data model	endpoint signatures; entities and keys
15-25	high-level design	boxes and arrows for each flow, labelled with QPS
25-40	deep dives (1-2)	the hard component, alternatives, failure modes
40-45	wrap-up	bottlenecks, what breaks at 10x, what you would monitor

The allocation is deliberately generous to requirements and estimation. They take ten minutes and determine every later decision. Candidates who skip them spend the deep dive defending choices they cannot justify.

Requirements: write them down

State requirements as a written list on the board, in the interviewer's words where possible. Then add the non-functional ones yourself - interviewers often leave them out on purpose:

- **Scale:** daily active users, requests per user per day, data size per item.
- **Latency:** which flow is user-facing and what p99 it needs.
- **Consistency:** which reads may be stale, and by how much.
- **Durability:** which data must never be lost.
- **Availability:** which flows must keep working during a partial outage.

Then state two or three **non-goals**. "Custom domains and link analytics dashboards are out of scope; I'll mention where analytics would attach." Non-goals protect your time and show judgement.

The estimation toolkit

Estimation is arithmetic with honest rounding. Memorise a few anchors and derive everything else:

TEXT		
seconds per day	86,400	~ 1e5 (round up; errs toward over-provisioning)
seconds per 30-day month	2,592,000	~ 2.6e6
1 million per day		~ 12 per second
1 billion per month		~ 386 per second
peak multiplier		2x-10x average, depending on diurnal shape

Every case in this book shows its arithmetic, and the companion model recomputes it. Take the URL shortener in chapter 2:

TEXT		
new links	100,000,000 / month	-> 38.6 writes/s average
reads	100 per write	-> 3,858 reads/s average
records	5 years	-> 6,000,000,000
storage	500 bytes each	-> 3.0 TB

Four numbers, two minutes. They already say that this is a read-heavy, small-data system: 3 TB fits on a single modern machine, so sharding is a question of throughput and availability, not of storage. That observation changes the rest of the design, and it is only available to a candidate who did the arithmetic.

Rounding rule: round in the direction that makes the design safer - up for load and storage - and say so. A factor-of-two error is fine; a factor-of-a-thousand error, such as confusing per-day with per-second, is disqualifying.

Choosing the deep dive

Every prompt has one component where the difficulty lives. Find it by asking which requirement is hardest to meet at the estimated load:

Prompt	Where the difficulty lives
URL shortener	key generation without collisions or coordination
Distributed cache	partitioning, rebalancing, hot keys
News feed	fan-out cost with a skewed follower graph
Chat	ordered delivery and connection state
Typeahead	top-k freshness under a write-heavy query log
Ride matching	proximity search on moving points
Payments and booking	exactly-once effects on money and inventory
Metrics pipeline	ingest volume and storage over time

Propose your deep dive, and let the interviewer redirect you. "The interesting part is key generation; I'd like to go deep there unless you'd rather I look at the redirect path." That sentence is itself an SDE-2 signal.

Labelling the diagram

Label every arrow with its request rate and every store with its size. An unlabelled diagram cannot be reasoned about; a labelled one lets the interviewer see immediately that the 3,858 reads per second go through a cache with a stated hit rate, and that the database behind it sees only the misses.

What an SDE-2 answer adds

For each component you place, answer three questions without being asked:

- 1 **Why this and not the obvious alternative?** "A key-value store, because every read is a point lookup by key; a relational database would work, but nothing here needs a join."
- 2 **What happens when it fails?** "If the cache cluster is lost, the database sees 3,858 reads per second, which one replica set can serve; latency rises, availability holds."
- 3 **What breaks first at 10x?** "The key-generation service, if it is a single counter; that is why I'd hand out ranges."

WATCH OUT | Failure modes and common mistakes

- **Boxes before numbers.** A design whose size is unknown cannot be defended.
- **Technology names as answers.** "Use Kafka" says nothing unless you say what ordering, retention, and consumer model you need from it.
- **One-size consistency.** Treating every read as strongly consistent, or every read as eventually consistent. Say which data needs which.
- **Deep-diving the easy part.** Designing the load balancer in detail while the key-generation question goes unanswered.
- **No failure story.** Every stateful component needs a sentence on what happens when it is lost.
- **Silent assumptions.** Assuming a number without saying so; the interviewer cannot tell a deliberate assumption from a mistake.

TRY IT | Interview questions and model answers

Q: Design X. Where do you start? A: With requirements on the board: three to five functional ones, the non-functional ones including latency, consistency and durability, and two or three non-goals. Then a five-minute estimate of load, storage and read/write ratio, because those numbers choose the architecture.

Q: Why estimate at all if the numbers are rough? A: Because the order of magnitude decides the design. Forty writes per second and three terabytes means one database can hold everything and the problem is availability and key generation. Four hundred thousand writes per second means sharding is the first decision. Being wrong by a factor of two changes nothing; skipping the estimate can change everything.

Q: How do you pick what to go deep on? A: I ask which requirement is hardest at the estimated load and propose that component, then let the interviewer redirect me. For a feed, that is fan-out with a skewed follower graph. For payments, it is exactly-once effects.

Q: What would you monitor? A: The SLI for each user-facing flow - latency and error rate - plus the saturation signal of the component I expect to break first at 10x: cache hit rate, consumer lag, replication lag, or connection count.

TRY IT | Exercises

- 1 Write requirements, non-goals and a five-minute estimate for "design a pastebin" before reading chapter 2.
- 2 For each prompt in the deep-dive table, name the requirement that makes that component hard.
- 3 Take any past design you have built at work and label every arrow with its request rate. Which number surprised you?

RECAP | Chapter summary

A high-level design is data flows sized against load, with a failure behaviour for each. Run a fixed 45-minute structure, write requirements and non-goals down, and estimate before drawing: the order of magnitude chooses the architecture. Spend the deep dive where the hardest requirement meets the load, and for every component say why, what happens when it fails, and what breaks first at 10x.

TRY IT | Revision checklist

- ☐ I can recite the 45-minute structure and its time budget.
- ☐ I can estimate QPS, storage and bandwidth from users and ratios in under five minutes.
- ☐ I state non-goals explicitly.
- ☐ I can name the hard component for each prompt in this book.
- ☐ For every component, I say why, how it fails, and what breaks at 10x.

SDE-2 CORE CHAPTER

Chapter 2 - URL Shortener: Key Generation and the Read Path

Learning objectives

By the end of this chapter, you should be able to:

- estimate the load and storage of a URL shortener and draw the conclusion the numbers support;
- compare random, hashed, and counter-based key generation with actual collision arithmetic;
- design a key-generation service that needs no coordination on the write path;
- choose between 301 and 302 redirects and explain what each costs; and
- size the cache on the read path and say what happens when it fails.

WHY IT WORKS | Why this matters at SDE-2

The URL shortener is the most common warm-up prompt, which makes it a strong filter: most candidates produce the same diagram, and the difference between a pass and a strong pass is entirely in the reasoning. Specifically, it is in key generation. "Hash the URL and take the first seven characters" is the answer most candidates give, and at the stated scale it produces millions of collisions. A candidate who computes that - and designs around it - has shown the core SDE-2 skill of letting numbers drive the design.

WHY IT WORKS | First-principles model

A URL shortener is a **globally unique key allocator in front of a read-heavy key-value lookup**.

- The **write path** allocates a short, unique key and stores `key -> long URL`.
- The **read path** looks the key up and redirects.
- The **invariant** is that a key, once issued, always points to the same URL. Handing out a key twice silently redirects someone's link to a stranger's URL - a correctness and security failure, not a performance one.

Core terminology

- **Short code:** the key in the short URL, such as `2crtgcg`.
- **Base62:** encoding with digits, lowercase, and uppercase letters - 62 symbols, URL-safe without escaping.
- **Key space:** the number of distinct codes of a given length; 62^7 for seven characters.
- **Birthday bound:** the collision count for random keys, approximately $n^2 / (2N)$ for n keys drawn from a space of N .
- **Key range allocation:** handing each application server a block of counter values to use without further coordination.
- **301 / 302:** permanent and temporary HTTP redirects.

Detailed mechanics

Requirements and estimate

In scope: create a short link for a long URL; redirect a short link; links do not expire by default. Out of scope: custom aliases (mentioned as an extension), analytics dashboards, and user accounts.

Non-functional: redirects are the hot path and need low latency; a key must never point to the wrong URL; created links must survive the loss of any single machine.

The companion computes the envelope:

TEXT

```
writes/s 38.6 average, reads/s 3,858 average
5 years -> 6,000,000,000 records, 3.0 TB at 500 bytes each
```

The conclusion to say out loud: **this is a small-data, read-heavy system**. Three terabytes fits on one machine; throughput is modest. The design problems are key generation, availability, and read latency - not storage scale.

Key generation: three options, with arithmetic

Option 1: random or hashed keys, truncated to seven characters. Seven base62 characters give $62^7 = 3.52 \times 10^{12}$ codes, which sounds inexhaustible. But collisions follow the birthday bound, not the key-space size:

TEXT

```
random 7-char keys for 6,000,000,000 URLs: ~5,111,292 expected collisions
```

Five million collisions over five years, and every one must be detected (a check before insert) and retried. Hashing the long URL instead of drawing randomly has the same collision behaviour for different URLs, with one advantage - identical URLs map to the same key - that is also a disadvantage if two users should get separate links for analytics.

The birthday formula is worth trusting, and the companion checks it at a scale where it can be simulated:

TEXT

birthday check, 2,000 keys into 62^3 : predicted 8.39 colliding pairs, measured 8.41 over 400 trials

Option 2: a single counter, encoded in base62. No collisions by construction, and short codes:

TEXT

counter id 125,000,000,000 -> base62 "2crtgcg" (7 chars)
 $62^7 = 3.522e+12$ keys; a counter lasts 2,894 years at this rate

The problem is that a single counter is a single point of failure and a coordination point on every write.

Option 3: counter ranges. A small coordination service - a database row updated transactionally, or a consensus-backed store such as ZooKeeper or etcd - hands each application server a block of, say, 10,000 ids. The server allocates from its block in memory with no network call and fetches a new block when it runs low. At 38.6 writes per second, a block of 10,000 lasts over four minutes on a single server, so the coordination service sees a request every few minutes.

Trade-offs to state:

- A server that crashes loses the rest of its block. At 3.5 trillion codes that waste is irrelevant; say so rather than engineering around it.
- Sequential codes are guessable. If links must not be enumerable, either encrypt the counter with a keyed 64-bit permutation before encoding - still collision-free, because a permutation is one-to-one - or add a random suffix and accept a longer code.

Option 3 is the answer to lead with: no collisions, no per-write coordination, and a failure mode that costs only unused ids.

Data model and storage

TEXT

links: code (PK, string) | long_url (string) | created_at | owner_id (nullable)

Every read is a point lookup by primary key, and no query needs a join. A key-value or wide-column store fits; a relational database also works comfortably at this size. Choose by operational familiarity and say that the access pattern does not force the choice. Replicate across at least three nodes in separate failure domains for durability.

The read path

TEXT

client -> CDN / edge -> redirect service -> cache -> links store

At 3,858 average reads per second - perhaps 20,000 at peak - with a skewed popularity distribution, a cache in front of the store absorbs most reads. A modest cache holding the hottest few percent of links typically achieves

a high hit rate on a Zipf-like workload; state the assumption and size it from the measured hit rate rather than presenting a guessed number as fact.

What happens when the cache is lost? The store sees the full read rate. At these numbers a replicated store can serve that directly, so the failure raises latency rather than causing an outage. Say this: it is the difference between a cache for latency and a cache that the system depends on for survival.

301 or 302?

- **301 (permanent):** browsers and proxies may cache the redirect, so repeat visits never reach you. Lower load and lower latency, but you lose visibility into clicks and cannot change the destination.
- **302 (temporary):** every visit reaches the service. Higher load, full analytics, and the destination can be changed or the link disabled - which matters for abuse takedowns.

Most production shorteners use a temporary redirect for control and analytics. State the trade-off and choose.

Analytics without slowing the redirect

Clicks are recorded by emitting an event to a queue after the redirect response is sent, never by a synchronous database write on the hot path. A stream processor aggregates counts per link per time bucket. If the queue is down, the redirect still works; analytics are delayed or sampled.

WATCH OUT | Failure modes and common mistakes

- **Truncated hash without collision handling.** Millions of collisions at the stated scale.
- **Single global counter.** A coordination point and single point of failure on every write.
- **Check-then-insert race.** Two servers generating the same random key both check, both see it free, both insert. Use a unique constraint and retry on violation.
- **Synchronous analytics.** Click logging on the redirect path turns an analytics outage into a redirect outage.
- **301 without saying what it costs.** Losing takedown ability is a real product risk.

TRY IT | Interview questions and model answers

Q: How do you generate the short code? A: A base62-encoded counter, with ranges of ids handed to each server by a small coordination service so there is no per-write coordination. It cannot collide. Truncating a hash to seven characters looks simpler, but with six billion links the birthday bound gives about five million collisions, each needing detection and retry.

Q: Isn't a sequential code guessable? A: Yes. If enumeration matters, I apply a keyed 64-bit permutation to the counter before encoding. A permutation is one-to-one, so it stays collision-free while the codes stop being sequential.

Q: What happens if the cache goes down? A: The store sees the full read rate - around 4,000 per second on average. A replicated store can serve that, so latency rises but redirects keep working. I'd alert on cache hit rate and store latency.

Q: 301 or 302? A: 302 by default. A 301 lets browsers cache the redirect, which saves load but gives up click analytics and the ability to disable a malicious link. If load became the problem, I'd put a CDN in front before switching to 301.

TRY IT | Exercises

- 1 Recompute the estimate for 1 billion new links per month. What changes in the design, and what does not?
- 2 How many base62 characters does a random-key scheme need to keep expected collisions under one over five years at 100 million links per month?
- 3 Design custom aliases. How do they interact with counter-generated codes, and how do you stop a custom alias from later colliding with a generated one?

RECAP | Chapter summary

A URL shortener is a unique-key allocator in front of a read-heavy lookup. The estimate shows a small-data, read-heavy system, so the difficulty is key generation. Truncated hashes collide millions of times at the stated scale; counter ranges encoded in base62 cannot collide and need no per-write coordination. Cache the read path for latency, keep analytics off it, and choose the redirect status deliberately.

TRY IT | Revision checklist

- ☐ I can produce the estimate and state what it implies.
- ☐ I can apply the birthday bound and explain why seven characters is not enough for random keys at this scale.
- ☐ I can design counter-range allocation and describe its failure mode.
- ☐ I can make codes non-enumerable without reintroducing collisions.
- ☐ I can defend 302 over 301.

SDE-2 CORE CHAPTER

Chapter 3 - Distributed Cache and Key-Value Store

Learning objectives

By the end of this chapter, you should be able to:

- explain why `hash(key) mod N` fails when the cluster changes size, with the fraction of keys it moves;
- implement consistent hashing and show, with measurements, why virtual nodes are needed;
- choose replication factor and read/write quorums for a stated consistency requirement;
- handle hot keys, cache stampedes, and node failure; and
- say which of these belong in a cache and which in a durable key-value store.

WHY IT WORKS | Why this matters at SDE-2

"Design a distributed cache" or "design a key-value store" is asked because it exposes whether a candidate understands partitioning as a mechanism rather than a word. Most candidates say "consistent hashing". Far fewer can say what it fixes, why a naive ring is badly unbalanced, or what happens to a key that becomes a million times hotter than its neighbours. The numbers in this chapter are the ones to reach for.

WHY IT WORKS | First-principles model

A distributed key-value system answers two questions for every key: **which nodes own it**, and **how many of them must agree** before a read or write succeeds.

- **Ownership** is partitioning. The goal is even load and minimal data movement when nodes join or leave.
- **Agreement** is replication. The goal is a stated consistency level at an acceptable latency and availability cost.

A cache and a store use the same two mechanisms with different defaults: a cache can lose data and rebuild it from the source of truth, so it usually replicates little and favours availability; a store cannot, so it replicates and uses quorums.

Core terminology

- **Partitioning (sharding)**: assigning each key to a node.
- **Consistent hashing**: placing nodes and keys on a hash ring; a key belongs to the next node clockwise.
- **Virtual node**: one of many ring positions owned by a single physical node.
- **Replication factor (N)**: the number of nodes holding each key.
- **Quorum (R, W)**: the number of replicas that must answer a read or acknowledge a write.
- **Hot key**: a key whose traffic far exceeds the average.
- **Cache stampede**: many concurrent misses on one key all hitting the source at once.

Detailed mechanics

Why mod N fails

The simplest partitioning is `node = hash(key) mod N`. It balances well and is one line. The problem appears when `N` changes. The companion places 1,000,000 keys on 10 nodes and adds an eleventh:

TEXT

```
add an 11th node: ring moves 8.8% of keys, hash mod N moves 90.9%
ideal is 1/11 = 9.1%
```

With `mod N`, about 10 of every 11 keys change owner. For a cache, that is a near-total miss storm at the moment you added capacity - usually because you were already under load. For a store, it is a migration of almost the whole dataset.

Consistent hashing

Hash each node to a position on a ring of 64-bit values; hash each key onto the same ring; a key belongs to the first node clockwise from it. A `TreeMap<Long, String>` makes the lookup one `ceilingEntry` call:

JAVA

```
String owner(String key) {
    Map.Entry<Long, String> e = ring.ceilingEntry(hash(key));
    return e != null ? e.getValue() : ring.firstEntry().getValue();
}
```

Adding a node takes over only the arc between it and its predecessor, so only about $1/(N+1)$ of keys move - the companion measures 8.8% against an ideal of 9.1%.

Why virtual nodes are not optional

With one position per node, the arcs between positions are very uneven, and so is the load:

TEXT

```
1 virtual nodes each: busiest node holds 2.55x the mean
10 virtual nodes each: busiest node holds 1.61x the mean
100 virtual nodes each: busiest node holds 1.14x the mean
200 virtual nodes each: busiest node holds 1.12x the mean
```

With a single position each, one node carries two and a half times its fair share, so capacity planning must be done for the busiest node and most of the fleet sits idle. A hundred virtual nodes per physical node brings the worst case within about 15% of the mean. Virtual nodes also make heterogeneous hardware easy - a node with twice the memory gets twice the virtual nodes - and spread a failed node's load across many survivors instead of dumping it all on one neighbour.

An alternative used by Redis Cluster is a fixed number of **hash slots** (16,384), assigned to nodes by a table. It has the same property - adding a node moves only the slots reassigned to it - with an explicit, inspectable mapping instead of a hashed ring. Mention it as the same idea with a different bookkeeping trade-off.

Replication and quorums

For a durable store, replicate each key to the next **N** distinct physical nodes on the ring. Reads and writes then choose how many replicas to wait for:

TEXT

```
W + R > N    -> every read quorum overlaps every write quorum: reads see the latest acknowledged write
W + R <= N    -> reads may miss the latest write: eventual consistency, lower latency
```

With $N = 3, W = 2, R = 2$ gives read-your-latest-write behaviour while tolerating one slow or failed replica on each side. $W = 1, R = 1$ is fastest and most available, and gives no such guarantee. State the choice per use case: a session cache can take $W = 1, R = 1$; an account setting that the user just changed should not.

Quorums need two supporting mechanisms, which you should name: **read repair** (a read that sees disagreeing replicas writes the newest value back to the stale ones) and **hinted handoff** (a write whose target replica is down is held by another node and delivered when it returns). Conflict resolution - last-writer-wins by timestamp, or version vectors when concurrent writes must be merged - is the next question an interviewer will ask.

Hot keys

Consistent hashing balances *keys*, not *traffic*. One celebrity's profile or one flash-sale item can send a large share of all reads to one node. Remedies, in increasing complexity:

- **Local caching** in the application tier for a few seconds. Most effective and simplest; it trades a little staleness for a large drop in remote reads.
- **Key replication for reads:** store the hot key under `key#1 . . key#k` on different nodes and read from a random copy. Writes must update every copy.
- **Request coalescing:** at most one in-flight fetch per key per node; other requests for the same key wait for that result.

Detecting hot keys needs per-key counters sampled at the proxy or client - a count-min sketch keeps that bounded in memory.

Cache stampede

When a popular key expires, thousands of concurrent requests miss at once and all go to the database. Fixes:

- **Request coalescing** (single-flight), as above.
- **Early probabilistic refresh:** a request that finds a key close to expiry refreshes it with a small probability, so one request refreshes it before the crowd arrives.
- **Stale-while-revalidate:** serve the expired value while one request fetches the new one.

Eviction and memory

Each cache node evicts with LRU or an approximation of it (SD-04 chapter 4 covers the data structures). At fleet scale, the important numbers are the working set and the hit rate: size the fleet so that the working set fits, and alert when the hit rate drops, because a falling hit rate is the leading indicator of database overload.

WATCH OUT | Failure modes and common mistakes

- **mod N partitioning** in a system that will ever resize.
- **A ring without virtual nodes.** The busiest node carried 2.55 times the mean in the measurement above.
- **Quorum settings without a stated consistency goal.** `N`, `R` and `W` are answers to a requirement, not defaults.
- **Assuming consistent hashing solves hot keys.** It balances keys, not traffic.
- **Treating the cache as durable.** If the design fails when the cache is empty, the cache is a database without the guarantees of one.
- **No stampede protection** on popular keys with a TTL.

TRY IT | Interview questions and model answers

Q: How do you partition keys across cache nodes? A: Consistent hashing with virtual nodes. With `mod N`, adding an eleventh node to ten moves about 91% of keys; the ring moves about 9%. Without virtual nodes the ring is badly unbalanced - one node carried 2.55 times the mean in a million-key test - so I use around a hundred per physical node, which brought the worst case to about 1.14 times the mean.

Q: How do you make reads consistent in a replicated store? A: Choose quorums so that $R + W > N$. With three replicas, writing to two and reading from two guarantees overlap, so a read sees the latest acknowledged write, and each side still tolerates one slow replica. I'd add read repair and hinted handoff, and choose last-writer-wins or version vectors for conflicts.

Q: One key gets 100 times the traffic of any other. What happens? A: Its owning node saturates, because hashing balances keys, not traffic. I'd add a short-lived local cache in the application tier first, then replicate the hot key under several suffixed keys for reads if needed, with coalescing so a miss triggers only one fetch.

Q: A hot key expires and the database falls over. Why, and how do you prevent it? A: Every concurrent request missed at once and went to the database - a stampede. Single-flight coalescing, early probabilistic refresh, or serving stale data while one request revalidates all prevent it.

TRY IT | Exercises

- 1 Compute the fraction of keys that move when a 10-node `mod N` cluster grows to 12 nodes. Compare with a ring.
- 2 Choose `N`, `R` and `W` for a shopping cart, a user's session token, and a bank balance, and justify each.
- 3 Design hot-key detection with bounded memory, and say where in the request path it runs.

RECAP | Chapter summary

Partitioning decides ownership; replication decides agreement. `mod N` moves almost every key when the cluster changes; consistent hashing moves about $1/(N+1)$, but only balances load with many virtual nodes per physical node. Quorums with $R + W > N$ give reads that see the latest acknowledged write. Hashing balances keys, not traffic, so hot keys and stampedes need their own defences.

TRY IT | Revision checklist

- ☐ I can state the fraction of keys moved by `mod N` and by a ring when a node is added.
- ☐ I can explain why virtual nodes are needed, with the imbalance numbers.
- ☐ I can pick `N`, `R` and `W` for a stated consistency goal.
- ☐ I can list three hot-key defences and three stampede defences.

SDE-2 CORE CHAPTER

Chapter 4 - News Feed: Fan-Out and the Celebrity Problem

Learning objectives

By the end of this chapter, you should be able to:

- compare fan-out on write (push) and fan-out on read (pull) by the work each does per post and per feed read;
- quantify how a skewed follower graph concentrates fan-out cost in a tiny fraction of accounts;
- design the hybrid model and choose its threshold with arithmetic rather than instinct;
- design the timeline store, the ranking step, and pagination; and
- handle deletes, privacy changes, and newly followed accounts.

WHY IT WORKS | Why this matters at SDE-2

"Design Twitter" or "design the Instagram feed" is where interviewers test whether a candidate can reason about *skew*. The averages are easy and misleading. The design is decided by the tail: a handful of accounts with tens of millions of followers. The strongest answers put a number on that tail and let it pick the architecture.

WHY IT WORKS | First-principles model

A feed is a **per-user, time-ordered merge of posts from the accounts that user follows**. The only real question is *when* the merge happens:

- **Push (fan-out on write):** when an account posts, write the post id into the precomputed timeline of every follower. Reads are cheap - one lookup. Writes cost one insert per follower.
- **Pull (fan-out on read):** store posts only in the author's list. When a user opens the feed, fetch recent posts from every followed account and merge. Writes are cheap; reads cost one fetch per followed account.

Neither is right in general. The follower graph decides.

Core terminology

- **Timeline (home feed):** the list of post ids a user sees, newest or highest-ranked first.
- **Fan-out:** copying one post id into many timelines.
- **Write amplification:** timeline writes per post.
- **Zipf distribution:** a power law where the k -th most popular item has popularity proportional to $1/k$.
- **Hybrid fan-out:** push for ordinary accounts, pull for accounts above a follower threshold.
- **Cursor pagination:** continuing a feed from an opaque position instead of a page number.

Detailed mechanics

Requirements and the naive estimate

In scope: post, follow, read a home feed of recent posts from followed accounts, paginate. Out of scope: search, trends, direct messages. Non-functional: feed reads are the hot path and should be fast; a new post should appear in followers' feeds within seconds, not instantly; losing a post is not acceptable.

The companion builds a follower graph for 1,000,000 accounts where the account at popularity rank r has $10,000,000 / r$ followers - a Zipf shape, which is what real social graphs roughly look like at the top. If every account posts once a day and we push:

TEXT

one post per account per day, push model: 143,435,683 timeline writes/day
that is 1,660 writes/s on average

On average that is easy. The average is not the problem.

The tail decides

The same model, splitting accounts by follower count:

TEXT

pull for accounts over 100,000 followers:	99 accounts (0.010%)	remove 36% of fan-out writes
pull for accounts over 10,000 followers:	999 accounts (0.100%)	remove 52% of fan-out writes
pull for accounts over 1,000 followers:	9,990 accounts (0.999%)	remove 68% of fan-out writes

One account in a thousand is responsible for more than half of all fan-out writes. And the cost is not only volume but *latency*:

TEXT

top account's single post at 50,000 writes/s: 200 s to fan out

A single post from the top account needs ten million timeline inserts. Even at 50,000 inserts per second dedicated to it, the last follower sees it more than three minutes later, and meanwhile that work queues in front of every ordinary account's posts.

The hybrid model

Push for accounts below a follower threshold; pull for accounts above it.

- **On post:** if the author is below the threshold, enqueue a fan-out job that inserts the post id into each follower's timeline. Otherwise, write the post only to the author's own post list.
- **On feed read:** fetch the user's precomputed timeline, fetch recent posts from the few above-threshold accounts they follow, merge by time or score, and return a page.

The read cost of the pull side is bounded by how many celebrity accounts a user follows, which is small for almost everyone. The threshold is a tuning knob: lower it and you save more writes but add more merges per read. With the numbers above, a threshold of 10,000 followers pulls for 999 accounts and removes over half the write load - a good default to propose, with the arithmetic that justifies it.

The timeline store

A timeline is a bounded list of post ids per user - say the latest 800 - stored in a fast in-memory store keyed by user id. Store **ids, not post bodies**: a post edited or deleted later is then correct everywhere with one update, and timelines stay small. Hydrate ids into full posts from a post cache at read time.

Two refinements to mention:

- **Inactive users:** skip fan-out for users who have not opened the app in, say, 30 days, and rebuild their timeline by pulling when they return. This removes a large share of writes for free.
- **Bounded length:** trim timelines on write. Older history is served by pulling from post lists, which is rare.

Ranking

Chronological feeds merge by time. Ranked feeds fetch a few hundred candidates - from the timeline and the pulled celebrity posts - and score them with a model, then return the top page. Keep ranking in a separate service with its own latency budget and a fallback to chronological order if it times out. The feed must not fail because ranking did.

Pagination

Use cursor pagination: the response includes an opaque cursor encoding the last post's score or timestamp and id, and the next request continues from it. Page numbers break when new posts arrive between requests; cursors do not.

Deletes, privacy, and new follows

- **Delete:** mark the post deleted in the post store; hydration drops it. Optionally clean timelines lazily.
- **Account goes private or blocks a follower:** filter at hydration time against the current relationship, because timelines may still hold old ids.
- **New follow:** backfill the new account's recent posts into the follower's timeline asynchronously, or rely on the pull path until the next post.

WATCH OUT | Failure modes and common mistakes

- **Choosing push or pull without the follower distribution.** The tail, not the average, decides.
- **Fan-out as one synchronous loop.** Must be queued, batched, and retryable.
- **Storing post bodies in timelines.** Edits and deletes become a fan-out problem too.
- **Ranking on the critical path without a fallback.** A ranking outage becomes a feed outage.
- **Offset pagination** on a feed that changes while the user scrolls.

TRY IT | Interview questions and model answers

Q: Push or pull? A: Hybrid, decided by the follower distribution. In a Zipf-shaped graph of a million accounts, the 999 accounts with more than 10,000 followers cause 52% of all fan-out writes, and the top account's single post needs ten million inserts - over three minutes at 50,000 writes per second. So I push for ordinary accounts and pull for accounts above a threshold, merging the two on read.

Q: How does the read path work in the hybrid model? A: Fetch the user's precomputed timeline of post ids, fetch recent posts from the few above-threshold accounts they follow, merge by time or score, hydrate the ids into posts from a cache, and return a page with a cursor.

Q: A celebrity deletes a post. What changes? A: Only the post store: it is marked deleted and hydration drops it. Timelines hold ids, so nothing needs to be rewritten, and celebrity posts were never fanned out anyway.

Q: How do you avoid wasting writes on users who never open the app? A: Skip fan-out for users inactive beyond a window and rebuild their timeline by pulling when they return.

TRY IT | Exercises

- 1 Recompute the fan-out table for a graph where the top account has 100 million followers. Does the threshold change?
- 2 Estimate the read-side cost of the hybrid model for a user who follows 50 above-threshold accounts. How would you cap it?
- 3 Design the fan-out worker: batching, retries, and what happens if it crashes halfway through a ten-million-follower post.

RECAP | Chapter summary

A feed is a per-user merge of followed accounts' posts; push does the merge on write, pull on read. Follower graphs are heavily skewed - in the model, 0.1% of accounts cause over half of fan-out writes - so the answer is hybrid: push for most accounts, pull for the tail, merge on read. Store ids rather than posts, skip inactive users, keep ranking off the critical path, and paginate with cursors.

TRY IT | Revision checklist

- ☐ I can state the per-post and per-read cost of push and of pull.
- ☐ I can quantify how skew concentrates fan-out cost, with the 0.1% / 52% example.
- ☐ I can describe the hybrid read path end to end.
- ☐ I can explain why timelines store ids, and how deletes and privacy changes are handled.

SDE-2 CORE CHAPTER

Chapter 5 - Chat and Presence: Ordering, Delivery, and Connections

Learning objectives

By the end of this chapter, you should be able to:

- choose a transport for real-time delivery and size the connection tier;
- define what "ordered" means in chat, and implement it with per-conversation sequence numbers;
- deliver messages exactly once to the user even though the network delivers at least once;
- route a message to a recipient connected to a different server;
- design presence and state its detection latency and load; and
- handle offline users, multiple devices, and group chats.

WHY IT WORKS | Why this matters at SDE-2

"Design WhatsApp" or "design Slack" looks like a CRUD system with a socket. It is actually a distributed ordering and delivery problem attached to a very large number of long-lived connections. Interviewers probe three places: what ordering guarantee you give and how, how a message finds a user connected to another machine, and what the presence feature costs. Each has a concrete answer.

WHY IT WORKS | First-principles model

A chat system is **a durable, per-conversation ordered log, plus a routing layer that pushes new log entries to whichever connections currently belong to the conversation's members.**

- The **log** is the source of truth. A message is "sent" when it is durably appended to its conversation.
- **Delivery** is best-effort pushing over live connections, backed by the log: a client that missed something catches up by reading from its last known position.
- **Ordering** is defined per conversation. There is no meaningful global order across all conversations, and trying to provide one is expensive and pointless.

Core terminology

- **WebSocket:** a persistent, bidirectional connection over HTTP; the usual transport for chat.
- **Gateway (connection server):** a server that holds client connections and forwards messages.
- **Sequence number:** a per-conversation, monotonically increasing id assigned when a message is appended.
- **At-least-once delivery:** every message arrives, possibly more than once.
- **Idempotent receive:** processing the same message twice has the same effect as once.
- **Presence:** whether a user is currently online, and when they were last seen.
- **Heartbeat:** a periodic message proving a connection is alive.

Detailed mechanics

Requirements and connection sizing

In scope: one-to-one and group messages, delivery and read receipts, online presence, history on a new device.
Out of scope: voice and video, end-to-end encryption details (mention where keys live), and media processing.

Non-functional: messages must not be lost once acknowledged; within a conversation everyone sees the same order; delivery to online users within a second.

The connection tier is sized by concurrent connections, not requests. The companion's numbers for 10 million concurrent users:

TEXT

at 500,000 connections per gateway: 20 gateways before headroom

State the per-server figure as an assumption - it depends on memory per connection and kernel tuning - and add headroom so that losing a gateway does not overload the rest.

Ordering with per-conversation sequence numbers

When a message arrives, the service appends it to its conversation's log and assigns the next sequence number for that conversation. Everyone in the conversation then orders by that number. One writer per conversation - route each conversation to one partition, for example by hashing the conversation id onto a partitioned log such as Kafka - makes assignment trivial and removes clock skew from the question. Ordering by client timestamps is the classic mistake: device clocks disagree, so two users can see different orders.

At-least-once in, exactly-once to the user

Networks deliver at least once: a retried send or a reconnect can repeat a message, and pushes can arrive out of order. The client makes delivery exactly-once *as observed by the user* by delivering in sequence order and ignoring anything already delivered. The companion's receiver buffers early arrivals, drops duplicates, and notices gaps:

TEXT

```
arrivals [1, 2, 4, 5, 2, 3, 6] -> delivered [m1, m2, m3, m4, m5, m6], gap requested for [3]
```

Message 4 arrives before 3, so it is buffered and 3 is requested from the log. The duplicate 2 is ignored. When 3 arrives, 3, 4 and 5 are released in order.

Sends need the same protection in the other direction. The client attaches a client-generated message id; if it retries because the acknowledgement was lost, the server recognises the id and returns the original sequence number instead of appending a duplicate.

Routing across gateways

User A is connected to gateway 3; user B is connected to gateway 17. The message service needs to know where B is:

- A **connection registry** maps user id to the gateways holding that user's connections - one entry per device. Gateways write to it on connect and remove entries on disconnect, with a TTL so that a crashed gateway's entries expire.
- On a new message, the service looks up each recipient's gateways and forwards the message to them, typically through a per-gateway queue or a pub/sub channel.
- If a recipient has no live connection, nothing is pushed. The message is already in the log, and a push notification is sent through the mobile platform's notification service.

Offline users, multiple devices, history

Each device stores the last sequence number it has seen for each conversation. On reconnect it asks for everything after that position. Multiple devices are just multiple positions. A new device reads recent history from the log. This design makes "offline" the same code path as "missed a push".

Receipts

Delivery and read receipts are themselves small messages flowing back to the sender, recording the highest sequence number delivered or read per recipient. Store the high-water mark, not one receipt per message: "read up to 482" replaces 482 separate records.

Group chats

Small groups fan out on write like one-to-one chats. Very large groups or channels - thousands of members - behave like the feed problem in chapter 4: push only to members who are currently connected, and let everyone else pull from the log on open.

Presence

Presence looks simple and is often the most expensive feature by request count. The companion's numbers for 10 million online users:

TEXT

```
10,000,000 online users, heartbeat every 30s: 333,333 heartbeats/s  
offline after 3 missed beats: detected 60-90 s after disconnect
```

A third of a million heartbeats per second is more traffic than all the messages in many chat systems. Ways to cut it:

- Use the gateway's knowledge of the socket: a clean disconnect is detected immediately, and heartbeats only need to catch silent drops.
- Keep presence state in the gateway tier and publish only *changes*, not heartbeats.
- Deliver presence only to users who are looking at a relevant conversation or contact list, not to every contact of every user.

State the detection latency explicitly - "offline" appears 60 to 90 seconds after a silent drop with these settings - and let the product decide whether that is acceptable.

WATCH OUT | Failure modes and common mistakes

- **Ordering by client timestamp.** Clocks disagree; use a per-conversation sequence number.
- **Global ordering.** Unnecessary and expensive; order per conversation.
- **Treating the push as the source of truth.** A missed push must be recoverable from the log.
- **No client message id.** Retried sends create duplicates.
- **Registry entries without TTL.** A crashed gateway leaves users looking online and unreachable.
- **Presence broadcast to everyone.** The heartbeat rate alone can dominate the system.

TRY IT | Interview questions and model answers

Q: How do you guarantee message order? A: Per conversation. Each conversation is routed to one partition, which appends messages and assigns increasing sequence numbers; clients order by sequence number. Client timestamps are unreliable, and a global order across conversations is expensive and unnecessary.

Q: The network can duplicate and reorder messages. What does the user see? A: Each message exactly once, in order. The client buffers early arrivals, ignores sequence numbers it has already delivered, and asks the log for any gap. On the send side, a client-generated message id makes retries idempotent.

Q: How does a message reach a user connected to a different server? A: A connection registry maps each user to the gateways holding their connections, with TTL-based entries. The message service looks recipients up and forwards to those gateways. If there is no connection, the message waits in the log and a push notification goes out.

Q: What does presence cost? A: With 10 million online users and a 30-second heartbeat, 333,333 heartbeats per second. I'd rely on socket disconnects for clean exits, publish only presence changes, and send them only to users viewing a relevant screen. Detection of a silent drop takes 60 to 90 seconds with three missed beats; the product should agree to that number.

TRY IT | Exercises

- 1 Size the gateway tier for 50 million concurrent users at 200,000 connections per gateway, with enough headroom to lose one availability zone out of three.
- 2 Design read receipts for a 500-member group. What do you store, and what do you send to the sender?
- 3 Extend the companion's receiver so that it re-requests a gap only if the gap is still open after a timeout.

RECAP | Chapter summary

Chat is a per-conversation ordered log plus a routing layer to live connections. Assign sequence numbers per conversation on a single writer, deliver at least once, and make delivery exactly-once to the user with in-order buffering, duplicate suppression, and gap requests. Route through a TTL-backed connection registry, treat offline as "catch up from the log", and size presence explicitly - it can be the heaviest traffic in the system.

TRY IT | Revision checklist

- ☐ I can size the gateway tier from concurrent users.
- ☐ I can explain per-conversation sequence numbers and why client timestamps fail.
- ☐ I can walk through the receiver handling [1, 2, 4, 5, 2, 3, 6].
- ☐ I can describe cross-gateway routing and what happens when a gateway crashes.
- ☐ I can state the presence heartbeat load and detection latency.

SDE-2 CORE CHAPTER

Chapter 6 - Typeahead and Search: Tries, Top-K, and Bloom Filters

Learning objectives

By the end of this chapter, you should be able to:

- design a typeahead service whose query path does no ranking work at request time;
- separate the online serving path from the offline aggregation path, and state the freshness trade-off;
- partition a prefix index without creating hot shards;
- size a Bloom filter for a target false-positive rate and explain where one saves real work; and
- sketch an inverted index for full-text search and say what makes it hard to update.

WHY IT WORKS | Why this matters at SDE-2

Typeahead is a latency problem disguised as a data-structure problem. A candidate who builds a trie and walks it on every keystroke has built something correct and too slow. The SDE-2 move is to precompute: the answer for every popular prefix exists before anyone types it, and the serving path is a lookup. Search and typeahead also share a component that interviewers like to probe - the probabilistic filter that avoids expensive lookups for things that are not there.

WHY IT WORKS | First-principles model

Typeahead is a **precomputed map from prefix to its top-k completions, rebuilt periodically from query logs, and served from memory.**

- **Online path:** keystroke -> prefix -> lookup -> k suggestions. Budget: tens of milliseconds end to end, because it runs on every keystroke.
- **Offline path:** query log -> aggregate counts over a window -> compute top-k per prefix -> publish a new index.

Nothing on the online path counts, sorts, or ranks. All of that happened offline.

Core terminology

- **Trie (prefix tree):** a tree whose paths spell strings; every node represents a prefix.
- **Top-k per node:** the k highest-scoring completions stored at each prefix node.
- **Inverted index:** a map from each term to the list of documents containing it.
- **Bloom filter:** a bit array with k hash functions that answers "definitely not present" or "possibly present".
- **False positive rate:** the fraction of absent keys a Bloom filter reports as possibly present.
- **Freshness:** how long a new popular query takes to appear in suggestions.

Detailed mechanics

Requirements and estimate

In scope: suggest up to 10 completions for a typed prefix, ranked by popularity, with some personalisation mentioned as an extension. Out of scope: spelling correction beyond a mention, and the search results page itself.

Non-functional: p99 well under 100 ms end to end; suggestions may lag real query trends by minutes to an hour.

A useful framing for the estimate: with 100 million searches a day and about five keystrokes that trigger a suggestion request per search, the service sees about 500 million requests per day - roughly 5,800 per second on average and several times that at peak. The index itself is small: the set of distinct popular queries and their prefixes fits in memory on each serving node.

Precompute top-k at every node

Store the top 10 completions at every trie node, so a lookup is a walk of at most the prefix length, then a read of a stored list. In practice the trie is flattened into a key-value map from prefix string to its list, which is simpler to shard, replicate, and serve.

Limit which prefixes get entries: very short prefixes are served from a small, heavily cached table, and prefixes with too little traffic are not stored at all.

The offline path and freshness

- 1 Query logs stream into a log store.
- 2 A batch or streaming job counts queries over a sliding window, with decay so that yesterday's spike fades.
- 3 The job computes the top-k per prefix and writes a new index version.
- 4 Serving nodes load the new version and switch atomically.

Freshness is a product decision. A job every 15 minutes means a breaking-news query takes up to 15 minutes, plus build time, to appear. If that is too slow, add a small real-time layer that tracks trending queries over the last few minutes and merges them into the stored suggestions at serving time.

Filtering belongs in this path too: remove offensive, private, or legally blocked queries before they reach the index, not at serving time.

Partitioning the prefix index

Partitioning by first letter is the obvious answer and a bad one: the traffic for "s" is many times the traffic for "x". Better options:

- **Replicate the whole index** on every serving node if it fits in memory. This is often the right answer, and it removes the partitioning question entirely.
- **Partition by hash of the prefix** when it does not fit. Balanced, and each request touches one shard, because the full prefix is known.

Bloom filters: saving expensive lookups

A Bloom filter answers set-membership with no false negatives and a tunable false-positive rate, using a fraction of the memory of storing the keys. The standard sizing for n items and target rate p :

TEXT

```
m = -n ln(p) / (ln 2)^2      bits
k = (m / n) ln 2             hash functions
```

The companion sizes one for 100,000 keys at 1%, inserts them, and measures:

TEXT

```
m = 958,506 bits (117 KB), k = 7 hashes, predicted FP 1.004%
measured on 200,000 non-members: 1.013%; 0 false negatives on 100,000 members
```

The measurement matches the formula, and there are no false negatives, which is the property that makes the structure useful. It uses about 9.6 bits per key regardless of how long the keys are.

Where it saves work in these systems:

- **LSM-tree storage** (Cassandra, RocksDB, HBase) keeps one filter per on-disk file, so a read for an absent key skips files without touching disk.
- **Web crawlers** check "have we already queued this URL?" without a database round trip.
- **Search and feed systems** avoid a remote lookup for items known to be absent, such as "has this user already seen this post?"

Two limitations to state: a standard Bloom filter cannot delete (a counting Bloom filter or a cuckoo filter can), and the false-positive rate climbs quickly once you insert more than the n it was sized for.

Full-text search in one paragraph

Search results use an inverted index: for each term, the list of document ids that contain it, sorted, with positions for phrase queries. A query intersects the lists for its terms and ranks the matches. Updates are the hard part - rewriting long posting lists on every change is too expensive - so search engines append new documents into small new segments and merge segments in the background, which is the same log-structured idea as an LSM tree. Near-real-time search means a new document is searchable once its segment is opened, typically within a second.

WATCH OUT | Failure modes and common mistakes

- **Walking and ranking on every keystroke.** Precompute top-k per prefix.
- **Partitioning by first letter.** Traffic per letter is extremely uneven.
- **No freshness story.** Say how long a new trend takes to appear, and how to shorten it if needed.
- **Filtering at serving time only.** Offensive or private queries should never enter the index.
- **Bloom filter without sizing.** Overfill it and the false-positive rate quietly climbs.
- **Expecting deletes from a standard Bloom filter.** Use a counting or cuckoo filter.

TRY IT | Interview questions and model answers

Q: How do you make typeahead fast? A: Precompute the top 10 completions for each prefix offline from the query logs, and serve lookups from an in-memory map on each node. Nothing is counted or sorted at request time. If the index fits in memory, I replicate it to every node rather than partitioning.

Q: How fresh are the suggestions? A: As fresh as the rebuild interval plus build time - say 15 to 30 minutes with a 15-minute job. If trending queries need to appear faster, I add a small streaming layer that tracks the last few minutes and merges its results in at serving time.

Q: Where would you use a Bloom filter, and how big is it? A: In front of any lookup that is expensive when the key is absent - LSM-tree files, crawler URL de-duplication, "already seen" checks. For 100,000 keys at 1%, it is about 958,000 bits - 117 KB - with 7 hash functions; I measured 1.01% false positives and no false negatives.

Q: How does a search index handle updates? A: New documents go into small new segments, which are merged in the background. Posting lists are never rewritten in place, which keeps writes cheap and makes new documents searchable within about a second.

TRY IT | Exercises

- 1 Size a Bloom filter for 1 billion URLs at 0.1% false positives. How much memory, and how many hash functions?
- 2 Add personalisation to typeahead - the user's own recent searches - without breaking the no-ranking-at-request-time rule.
- 3 Estimate the size of a prefix index for 10 million distinct popular queries averaging 20 characters, storing 10 completions per prefix.

RECAP | Chapter summary

Typeahead precomputes the top-k completions for every popular prefix offline and serves them from memory; freshness is the rebuild interval, and a small streaming layer can shorten it. Replicate the index if it fits, and hash-partition prefixes if it does not. Bloom filters answer "definitely not present" at about 9.6 bits per key for 1% false positives, which is why storage engines and crawlers use them to avoid expensive lookups.

TRY IT | Revision checklist

- ☐ I can describe the online and offline paths of typeahead separately.
- ☐ I can explain why first-letter partitioning fails.
- ☐ I can size a Bloom filter from n and p , and name three places it saves work.
- ☐ I can explain how an inverted index takes updates without rewriting posting lists.

SDE-2 CORE CHAPTER

Chapter 7 - Proximity and Ride Matching: Geohash and Moving Points

Learning objectives

By the end of this chapter, you should be able to:

- explain why a two-column latitude/longitude index cannot answer "nearby" efficiently;
- encode locations as geohashes and choose a precision from the cell sizes;
- handle the boundary problem that makes a single-cell lookup wrong;
- design a location service for frequently moving drivers, sized by update rate; and
- design the matching step so that one driver is never assigned to two riders.

WHY IT WORKS | Why this matters at SDE-2

"Design Uber", "design Yelp", and "find nearby friends" all reduce to the same question: given a point, find the things near it, fast, while those things may be moving. The interviewer is checking whether you know a spatial indexing technique, whether you know its boundary flaw, and whether you notice that a moving fleet makes this a write-heavy problem rather than a read-heavy one.

WHY IT WORKS | First-principles model

Proximity search needs a **one-dimensional key that keeps nearby points close together**, so that an ordinary sorted index or hash map can find neighbours.

- **Static places** (restaurants): written rarely, read often. Any spatial index works; cache aggressively.
- **Moving drivers**: written every few seconds per driver, read on every ride request. The index must absorb a high update rate, and exact durability of every position does not matter - the next update replaces it.
- **Matching** is a separate, transactional step: choose among the nearby candidates and reserve exactly one.

Core terminology

- **Geohash:** a base32 string made by interleaving the bits of longitude and latitude; shared prefixes mean nearby cells.
- **Precision:** geohash length; each extra character shrinks the cell by a factor of 32.
- **Boundary problem:** two points metres apart can have geohashes with a short common prefix.
- **Quadtree:** a tree that splits space into four quadrants, subdividing dense areas more finely.
- **H3 / S2:** hierarchical hexagonal (H3) and spherical (S2) cell systems used in production.
- **Location update:** a driver's periodic position report.

Detailed mechanics

Why two B-tree columns are not enough

An index on latitude finds everything in a horizontal band - a strip across the whole planet. An index on longitude finds a vertical band. The database can intersect them, but each band may contain millions of rows, so "nearby" becomes an expensive scan. The fix is a key that encodes both dimensions together.

Geohash

Geohash alternately halves the longitude and latitude ranges, recording one bit per halving, and emits one base32 character per five bits. Every extra character narrows the cell. The companion computes cell sizes at the equator:

TEXT	
precision	cell at the equator (width x height)
4	39.1 km x 19.6 km
5	4.9 km x 4.9 km
6	1.2 km x 611 m
7	153 m x 153 m
8	38 m x 19 m

and encodes a real point:

TEXT
San Francisco (37.7749, -122.4194) -> 9q8yyk8y

The rule for choosing: **pick the finest precision whose cells are at least as large as the search radius in both directions.** Only then does the rider's cell plus its eight neighbours (next section) cover every point within the radius. For "drivers within 500 metres", precision 6 works: its cells are at least 611 m on each side. For "within a kilometre", precision 6 is too fine - a cell is only 611 m tall - so use precision 5, or keep precision 6 and search a 5 x 5 block. Cells also shrink in width away from the equator, because lines of longitude converge, so choose precision by the latitude you serve.

The boundary problem

Geohash puts nearby points in the same cell *most of the time*. At a cell edge it does not:

TEXT

two points 5.3 m apart across a cell edge: 9q8yyk vs 9q8yys (5 shared chars)

Two points five metres apart fall into different precision-6 cells. A search that looks only in the rider's own cell misses the driver across the street. The standard fix is to search the rider's cell **and its eight neighbours**, then filter the candidates by true distance. Neighbour cells are computed directly from the geohash, so this is nine lookups, not a scan - and it is only complete if the precision follows the rule above, so that no point within the radius lies beyond the neighbouring cells.

Alternatives worth naming

- **Quadtree:** subdivides dense areas more finely, so each leaf holds a bounded number of points. Good for static data with very uneven density; harder to update at high write rates.
- **S2 and H3:** production-grade cell systems. S2 maps the sphere onto a cube and uses a space-filling curve; H3 uses hexagons, whose neighbours are all the same distance away, which simplifies "ring around this cell" searches. Mention that real ride-hailing systems use systems like these; the geohash reasoning carries over.

The location service

In scope: drivers report location; riders request a ride; the system finds nearby available drivers and assigns one. Out of scope: pricing, routing, and payments (chapter 8).

The write rate is the design driver. With, say, 1 million active drivers reporting every 4 seconds, the service absorbs 250,000 location updates per second. Each update:

- 1 goes to an in-memory store keyed by driver id, holding the latest position and status;
- 2 moves the driver between cell sets if the geohash cell changed - most updates do not change cell at precision 6, which keeps index churn low;
- 3 is appended to a stream for analytics and trip records, off the hot path.

Positions are soft state: if a node is lost, drivers re-report within seconds. Do not put every location update into a durable transactional database.

Partition the in-memory index by geography - by coarse cell or by city - so that a ride request touches one partition and its neighbours. Dense cities need finer partitions than rural areas; that unevenness is the case for a quadtree-like split or for sizing partitions by load rather than by area.

Matching without double assignment

A ride request finds candidates in the rider's cell and neighbours, filters by true distance, and ranks by estimated time to pickup. Then it must reserve one driver. Two riders may pick the same driver at the same moment, so reservation is a conditional update - the same check-then-act pattern as seat booking in chapter 8:

TEXT

```
set driver.status = RESERVED, driver.trip = T only if driver.status == AVAILABLE
```

If the update fails, take the next candidate. Offer the trip to the driver with a timeout; if the driver declines or the timeout fires, release the reservation and try the next one.

WATCH OUT | Failure modes and common mistakes

- **Latitude and longitude as two independent indexes.** Each one returns a planet-wide band.
- **Searching only the rider's cell.** Misses drivers just across an edge.
- **Durable storage for every location update.** Position is soft state; the write rate makes this expensive for no gain.
- **Choosing precision without checking it against the radius.** If a cell is smaller than the search radius, the 3 x 3 search misses points; precision 5 is about 5 km, precision 7 about 150 m.
- **Assignment without a conditional update.** Two riders get the same driver.

TRY IT | Interview questions and model answers

Q: How do you find drivers near a rider? A: Geohash the positions and index drivers by cell, at the finest precision whose cells are at least the search radius on each side. For a 500-metre radius that is precision 6, whose cells are about 1.2 km by 0.6 km at the equator. Query the rider's cell and its eight neighbours, then filter by true distance.

Q: Why the neighbours? A: Because nearby points can fall in different cells. In the companion, two points 5 metres apart straddle a precision-6 edge and share only five characters. Without the neighbours you miss the closest driver.

Q: What is the hardest load in this system? A: Location updates. A million drivers every four seconds is 250,000 writes per second. I keep positions in partitioned in-memory stores as soft state, update the cell index only when a driver changes cell, and stream updates for analytics off the hot path.

Q: How do you stop two riders getting the same driver? A: Reservation is a conditional update from available to reserved. The second request's update fails, so it takes the next candidate.

TRY IT | Exercises

- 1 Pick a precision for "restaurants within 200 metres" and say how many cells a query touches.
- 2 Estimate memory for the driver location store: 1 million drivers, with id, position, status, and cell.
- 3 Design surge detection: counting requests and available drivers per cell per minute.

RECAP | Chapter summary

Proximity search needs a key that encodes both dimensions; geohash provides one, with precision chosen from cell size. Nearby points can straddle a cell edge, so search the cell and its eight neighbours and filter by distance. A moving fleet makes location a write-heavy, soft-state problem sized by update rate. Matching reserves a driver with a conditional update so that no driver is assigned twice.

TRY IT | Revision checklist

- ☐ I can explain why two independent indexes cannot answer "nearby".
- ☐ I can pick a geohash precision so that the 3 x 3 search covers the radius.
- ☐ I can demonstrate the boundary problem and its fix.
- ☐ I can size the location-update rate and explain why positions are soft state.
- ☐ I can reserve a driver without double assignment.

SDE-2 CORE CHAPTER

Chapter 8 - Payments and Ticket Booking: Exactly-Once Effects

Learning objectives

By the end of this chapter, you should be able to:

- make a payment request safe to retry with an idempotency key, and state what the key must cover;
- record money movements in a double-entry ledger and assert its invariant;
- integrate an external payment provider whose calls may time out with an unknown result;
- keep the order database and the event stream consistent with a transactional outbox;
- design seat inventory so that no seat is sold twice under a flash-sale load; and
- reconcile your records against the provider's.

WHY IT WORKS | Why this matters at SDE-2

Payments and booking are where "eventually consistent" stops being an acceptable answer. Charging a customer twice, or selling one seat to two people, is a visible, costly failure. Interviewers use these prompts to see whether a candidate knows that networks deliver at least once, that exactly-once is built from idempotency plus deduplication, and that the hard part is the timeout whose outcome you do not know.

WHY IT WORKS | First-principles model

Exactly-once **delivery** is impossible over an unreliable network. Exactly-once **effects** are achievable: deliver at least once, and make every effect idempotent so that repeats do nothing.

Three tools do all the work in this chapter:

- 1 **Idempotency keys** make repeated requests return the original result.
- 2 **A double-entry ledger** makes every money movement balance, so that errors are detectable.
- 3 **Conditional updates** make contended inventory changes atomic.

Core terminology

- **Idempotency key:** a client-generated id for one logical operation; the server stores the result under it.
- **Double-entry ledger:** every transaction is a set of entries across accounts that sum to zero.
- **Payment service provider (PSP):** the external system that actually moves money (card networks, wallets).
- **Transactional outbox:** writing an event to an outbox table in the same database transaction as the state change, then publishing it asynchronously.
- **Saga:** a sequence of local transactions with compensating actions on failure.
- **Reconciliation:** comparing internal records with the provider's settlement reports.
- **Hold (reservation):** a temporary claim on inventory that expires unless confirmed.

Detailed mechanics

Idempotency keys

The client generates a key per logical payment - typically per checkout attempt - and sends it with every retry. The server stores the key with the result. A repeated key returns the stored result without charging again.

The companion replays 100 orders where each request is delivered one to four times:

TEXT

```
100 orders, 238 charge requests (retries included) -> 100 charges  
merchant revenue $1,999.00, every transaction sums to zero: true
```

Details that matter:

- **Scope the key to the operation, not the request.** A new key per HTTP retry defeats the purpose.
- **Store the key and the effect atomically.** If the charge is recorded and the key is not, a retry charges again. In one database, write both in one transaction; the unique constraint on the key is what makes concurrent duplicates safe.
- **Bind the key to the request body.** A reused key with a different amount is an error, not a cache hit.
- **Keep keys long enough** to cover every client's retry window - days, not minutes.

The double-entry ledger

Never store a balance as a single mutable number that you add to and subtract from. Record every movement as a transaction with at least two entries that sum to zero:

TEXT

```
txn pay-17:  customer:c3      -1,999  
             merchant:revenue +1,999
```

A balance is the sum of an account's entries (cached for speed). The invariant - every transaction sums to zero, so the whole ledger sums to zero - is checkable at any time, and the companion checks it after every charge. A bug that creates or destroys money shows up as a non-zero sum, rather than as a customer complaint weeks later. Entries are append-only; a refund is a new transaction, never an edit.

The external provider and the unknown outcome

The hardest case: you call the provider to charge a card and the call times out. The charge may or may not have happened.

- **Never assume failure and retry blindly with a new request.** That is how customers get charged twice.
- **Send your idempotency key to the provider**, so that a retry with the same key is safe on their side too; most providers support this.
- **Record a `PENDING` state before the call**, and resolve it afterwards: by the call's response, by a webhook from the provider, or by querying the provider's status for your key.
- **A reconciler** periodically resolves payments stuck in `PENDING` by querying the provider.

The state machine is short and every edge is explicit: `CREATED` -> `PENDING` -> `SUCCEEDED` | `FAILED`, with `PENDING` resolved only by evidence from the provider.

Keeping the database and the event stream consistent

After a payment succeeds, other services need to know - to ship the order, send a receipt, update analytics. Writing to the database and then publishing to a queue is two operations; a crash between them loses the event, and publishing first can announce a payment that never committed.

The **transactional outbox** fixes this: in the same database transaction as the state change, insert an event row into an `outbox` table. A separate relay reads the outbox and publishes to the queue, marking rows as sent. Publishing is at-least-once, so consumers deduplicate by event id. SD-02 covers the pattern in depth; here, name it and place it.

Ticket booking: contended inventory

Booking has the same exactly-once concerns plus heavy contention on a small set of rows during a popular sale.

The core operation is a conditional update on the seat:

TEXT

```
UPDATE seats SET state = 'HELD', hold_id = ?, hold_expires = now() + 10 min
WHERE event_id = ? AND seat_id = ? AND state = 'AVAILABLE'
```

One affected row means the hold succeeded; zero means someone else got it. The companion runs the in-memory equivalent - 50 buyers racing for 10 seats with compare-and-set:

TEXT

```
50 buyers race for 10 seats with compare-and-set: 10 booked, 10 distinct holders
```

The rest of the design follows from the flow:

- **Hold, then pay, then confirm.** A seat is held while the customer pays; confirmation turns the hold into a booking. Expired holds return to available - lazily when read, and with a periodic sweep.
- **Payment is the idempotent operation from earlier in this chapter**, keyed by the hold id, so a retried checkout cannot pay twice for one hold.
- **Flash-sale load:** put a virtual waiting room in front of the booking service so that it admits users at a rate the inventory database can serve, rather than letting a million requests contend for the same rows at once.
- **General admission** (a count, not specific seats) is a conditional decrement - `UPDATE ... SET remaining = remaining - n WHERE remaining >= n` - or a pre-split set of counters across shards when one row is too hot.

Reconciliation

Once a day at least, compare your ledger with the provider's settlement report, line by line. Mismatches - a charge they have that you do not, an amount that differs - go to a queue for investigation. Reconciliation is the backstop that catches everything the rest of the design missed, and financial systems are expected to have one.

WATCH OUT | Failure modes and common mistakes

- **Retrying a timed-out charge without an idempotency key.** Double charges.
- **A mutable balance column.** Errors are silent; use a ledger with a zero-sum invariant.
- **Key and effect stored in separate transactions.** A crash between them re-enables duplicates.
- **Dual write to database and queue.** Lost or phantom events; use an outbox.
- **Check-then-update seat booking.** Double-sold seats; use a conditional update.
- **Holds that never expire.** Inventory leaks until someone notices.
- **No reconciliation.** Discrepancies are found by customers or auditors instead.

TRY IT | Interview questions and model answers

Q: How do you avoid charging a customer twice? A: The client sends an idempotency key per checkout attempt, on every retry. The server stores the key and the result in the same transaction as the charge, under a unique constraint, and returns the stored result for a repeated key. I pass the same key to the payment provider so that retries are safe on their side too.

Q: The provider call times out. Was the customer charged? A: Unknown, so the payment stays **PENDING**. It is resolved by the provider's webhook or by querying the provider for our idempotency key - never by assuming it failed and charging again. A reconciler sweeps anything left pending.

Q: How do you store money movements? A: As a double-entry ledger: each transaction is entries that sum to zero, balances are sums of entries, and entries are append-only. The zero-sum invariant is checked continuously, so a bug that creates or destroys money is detected.

Q: How do you prevent a seat being sold twice in a flash sale? A: A conditional update from **AVAILABLE** to **HELD**, so only one request can win. Holds expire after a few minutes. In front of it, a waiting room admits users at a rate the database can handle, so that a million people are not contending for the same rows at once.

TRY IT | Exercises

- 1 Write the full state machine for a payment, including refunds and partial refunds, and list which transitions are triggered by the provider.
- 2 Design idempotency-key storage for 10 million payments per day with a seven-day retention.
- 3 For a stadium sale of 50,000 seats and 2 million interested users, design the waiting room: admission rate, fairness, and what users see.

RECAP | Chapter summary

Exactly-once effects come from at-least-once delivery plus idempotency. Idempotency keys, stored atomically with the effect and passed to the provider, make charges safe to retry; timeouts leave payments **PENDING** until evidence resolves them. A double-entry ledger makes money movements checkable; an outbox keeps state and events consistent. Seat inventory uses conditional updates and expiring holds, protected by a waiting room during flash sales, and reconciliation catches whatever is left.

TRY IT | Revision checklist

- ☐ I can list four rules for idempotency keys.
- ☐ I can explain the double-entry invariant and why it beats a balance column.
- ☐ I can handle a provider timeout without risking a double charge.
- ☐ I can explain the transactional outbox and why consumers still deduplicate.
- ☐ I can write the conditional update for a seat hold and describe the waiting room.

SDE-2 CORE CHAPTER

Chapter 9 - Metrics Pipeline and Object Storage: Volume and Durability

Learning objectives

By the end of this chapter, you should be able to:

- size a metrics pipeline by series count and resolution, and separate the effects of compression and downsampling;
- design ingestion, storage, and query paths for time-series data;
- explain cardinality as the failure mode of metrics systems;
- compare replication and erasure coding by storage overhead and loss probability; and
- design an object store's metadata and data paths.

WHY IT WORKS | Why this matters at SDE-2

"Design a monitoring system" and "design S3" look unrelated but share the same core: very large volumes of data whose cost is dominated by storage, where the design is decided by arithmetic about bytes rather than by request routing. Candidates who reason about requests per second and ignore bytes per day tend to produce designs that are correct and unaffordable.

WHY IT WORKS | First-principles model

Both systems separate **a small, strongly consistent metadata layer** from **a large, cheap, append-oriented data layer**.

- In a metrics system, metadata is the set of series and their labels; data is the stream of timestamped values.
- In an object store, metadata is the bucket, key, and location of each object's pieces; data is the bytes.

Durability and cost are decided in the data layer, by how many bytes you keep and how you protect them.

Core terminology

- **Time series:** a stream of timestamped values identified by a metric name and a set of labels.
- **Cardinality:** the number of distinct series, driven by label combinations.
- **Downsampling (rollup):** replacing fine-grained points with aggregates over coarser intervals.
- **Delta-of-delta and XOR encoding:** the compression scheme from Facebook's Gorilla paper, which stores timestamps and values as small differences.
- **Replication factor:** full copies of each object.
- **Erasure coding RS(k, m):** splitting data into k data fragments plus m parity fragments; any k of the $k + m$ reconstruct it.
- **Repair window:** the time between a node's failure and the restoration of its data elsewhere.

Detailed mechanics

Sizing a metrics system

Start from series and resolution, not from hosts. The companion's numbers for one million series scraped every 10 seconds:

TEXT

```
1,000,000 series every 10s: 8,640,000,000 points/day  
16 bytes/point raw: 138 GB/day; ~1.37 bytes/point compressed: 11.8 GB/day
```

Sixteen bytes is a 64-bit timestamp and a 64-bit value. The 1.37 bytes per point is the average the Gorilla paper (Pelkonen et al., VLDB 2015) reports for its delta-of-delta timestamp and XOR value encoding on production data. Your ratio depends on your data; state it as the published figure, not a guarantee.

Over a year, two levers matter:

TEXT

```
one year at 10s: 50.5 TB raw, 4.32 TB compressed  
compressed and tiered (10s for 14d, 1m for 90d, 1h for 1y): 0.36 TB  
compression saves 11.7x; tiering saves a further 12.2x
```

Compression and downsampling are separate and roughly equal levers here, and together they reduce a year of storage by more than a hundred times. Keeping every 10-second point for a year is rarely what anyone needs: dashboards over weeks use minute or hour rollups anyway.

Ingestion, storage, and query

TEXT

```
agents -> ingest gateways -> partitioned log (Kafka) -> TSDB writers -> blocks on disk / object store
                                     ^
                                query service -----+ (recent data from memory, old from blocks)
```

- **Ingest** validates, rate-limits per tenant, and appends to a partitioned log. The log decouples bursts from storage and allows replay.
- **Writers** keep the most recent hours in memory, compressed per series, and flush immutable blocks - for example every two hours - to local disk and then to object storage.
- **Compaction** merges small blocks into larger ones and produces the rollups.
- **Queries** fan out to the partitions holding the requested series and time range, reading recent data from memory and older data from blocks.

Partition by series - a hash of metric name and labels - so that each series' points land together, which is what makes compression effective.

Cardinality is the failure mode

The number of series is the product of label values. Add a `user_id` label to a request-latency metric and one metric becomes a million series; add a `request_id` label and it becomes unbounded. Memory in the writers grows with active series, so a single bad label can take the system down. Defences: per-tenant series limits enforced at ingest, rejecting or dropping labels that exceed a cardinality budget, and alerting on the rate of new series. Mention this without being asked: it is the most common real-world failure of metrics systems.

Replication or erasure coding?

For an object store, the durability question is: what is the probability that enough pieces of an object are lost before the repair process restores them? The companion assumes each node independently has a 1% chance of failing within one repair window:

TEXT

3x replication	overhead 3.00x	P(loss per object) 1.00e-06
RS(6,3) erasure	overhead 1.50x	P(loss per object) 1.21e-06
RS(10,4) erasure	overhead 1.40x	P(loss per object) 1.86e-07

Three observations, each worth saying in an interview:

- **RS(10,4) is about five times more durable than 3x replication at under half the storage.** For large, cold data, that is why object stores use erasure coding.
- **RS(6,3) is not automatically better than replication:** at this failure rate it halves the storage but is slightly less durable. The parameters matter, not the technique's name.

- **The repair window is the real lever.** The probability of loss rises steeply with the failure probability, and that probability is proportional to how long repair takes. Faster detection and parallel repair across many nodes improve durability more than adding parity.

Erasure coding has costs: reading a small object may touch k nodes, and repairing one lost fragment requires reading k others, so it suits large objects. Many systems replicate small or hot objects and erasure-code large or cold ones.

Independent failures are the model's big assumption. Real failures correlate - a rack, a power feed, a bad firmware rollout - so place fragments across failure domains, and say that the independent-failure numbers are an upper bound on reality.

The object store's two paths

- **Metadata:** bucket, key $\rightarrow$ object id, size, checksum, fragment locations, version. Strongly consistent, partitioned by bucket and key, and small relative to the data. Listing by prefix needs the metadata to be ordered by key within a bucket.
- **Data:** large objects are split into chunks, each chunk erasure-coded or replicated across storage nodes in different failure domains. Uploads are multipart, so a 50 GB upload can resume after a failure.
- **Integrity:** store a checksum per chunk, verify on read, and run a background scrubber that re-verifies data at rest and repairs silent corruption.

WATCH OUT | Failure modes and common mistakes

- **Sizing by hosts instead of series.** Series count and resolution determine cost.
- **Unbounded label cardinality.** One label can multiply series by millions.
- **Keeping full resolution forever.** Tiered rollups cut storage by an order of magnitude.
- **Assuming erasure coding always beats replication.** Compare loss probability for the actual parameters.
- **Ignoring the repair window and correlated failures.** Both dominate real durability.
- **No checksums or scrubbing.** Silent corruption goes undetected until a read fails.

TRY IT | Interview questions and model answers

Q: How much storage does a metrics system need? A: From series and resolution. A million series at 10 seconds is 8.64 billion points a day - 138 GB raw at 16 bytes per point, about 12 GB with Gorilla-style compression. For a year, tiered rollups - 10 seconds for two weeks, one minute for 90 days, one hour for a year - bring it from 4.3 TB compressed to about 0.36 TB.

Q: What breaks a metrics system in practice? A: Cardinality. A label like a user id turns one metric into millions of series, and writer memory grows with active series. I enforce per-tenant series limits at ingest and alert on new-series rate.

Q: Replication or erasure coding for an object store? A: Depends on object size and the parameters. With a 1% chance of a node failing within the repair window, RS(10,4) loses an object with probability about $2e-7$ at

1.4x overhead, against 1e-6 at 3x for triple replication - so for large objects erasure coding wins on both. RS(6,3) is slightly less durable than 3x at that rate, so the parameters matter. Small or hot objects I'd replicate, because erasure coding makes small reads and repairs expensive.

Q: How do you protect against silent corruption? A: Checksums per chunk, verified on every read, and a background scrubber that re-reads and repairs data at rest.

TRY IT | Exercises

- 1 Recompute the metrics storage for 20 million series at 15-second resolution with the same tiering.
- 2 Compute the loss probability for RS(12,4) and for 2x replication at a 1% failure rate. Which would you use for a backup tier?
- 3 Design `LIST bucket prefix=logs/2026/` over a metadata store partitioned by key. What partitioning makes it efficient?

RECAP | Chapter summary

Metrics systems and object stores split a small, consistent metadata layer from a large data layer whose cost is bytes. Size metrics by series and resolution: compression and tiered rollups each cut storage by roughly an order of magnitude, and cardinality is the failure mode to guard. For durability, compare actual loss probabilities: RS(10,4) beats triple replication at under half the storage, RS(6,3) does not, and the repair window and correlated failures matter more than the choice of technique.

TRY IT | Revision checklist

- ☐ I can size points per day and storage per year from series and resolution.
- ☐ I can separate the savings from compression and from downsampling.
- ☐ I can explain cardinality and three defences.
- ☐ I can compare 3x, RS(6,3) and RS(10,4) with numbers.
- ☐ I can describe the metadata and data paths of an object store.

SDE-2 CORE CHAPTER

Chapter 10 - Case Bank, Follow-Ups, and Scoring

Learning objectives

By the end of this chapter, you should be able to:

- map an unfamiliar prompt onto the cases in this book by its dominant difficulty;
- answer the follow-up questions that close most design rounds;
- score your own mock rounds against a consistent rubric; and
- choose which case to practise next from your scores.

WHY IT WORKS | Why this matters at SDE-2

No list of prompts covers every interview. What transfers is recognition: most new prompts are one of a handful of difficulties in a new costume. "Design a leaderboard" is the cache and hot-key problem from chapter 3 with a sorted set. "Design Google Docs" is chat's ordering problem from chapter 5 with operational transforms or CRDTs. A candidate who recognises the difficulty in the first five minutes spends the rest of the round on the right thing.

WHY IT WORKS | First-principles model

Every prompt in this book is dominated by one of six difficulties:

Difficulty	Cases in this book	The question it forces
Unique allocation	URL shortener, seat booking	how to hand out ids or inventory without collisions or double-selling
Partitioning and skew	cache and KV store, feed	how to spread load when keys or users are extremely uneven
Ordering and delivery	chat, payments	how to get exactly-once effects in order over an unreliable network
Precomputation and freshness	typeahead, feed ranking	what to compute ahead of time, and how stale it may be
Spatial and moving data	ride matching	how to index points that are near each other and keep moving
Volume and durability	metrics, object storage	how many bytes, for how long, and how they survive failures

Identify the difficulty, then reuse the reasoning from the matching chapter.

Core terminology

- **Dominant difficulty:** the requirement that is hardest to meet at the estimated load.
- **Follow-up:** a question after the main design about failure, scale, or change.
- **Rubric:** the dimensions an interviewer scores, stated so that you can score yourself.

Detailed mechanics

Mapping new prompts

New prompt	Closest case	What changes
Pastebin	URL shortener	objects are larger: store bodies in an object store, keep metadata in the key-value store
Leaderboard	Cache and KV store	sorted sets per board; the top board is a hot key
Collaborative editor	Chat	ordering per document; concurrent edits need operational transforms or CRDTs
Notification system	Chat and feed	fan-out to devices; preferences and rate limits per user
Web crawler	Typeahead (Bloom filter) and metrics (volume)	URL frontier, politeness per host, de-duplication with a Bloom filter
Distributed job scheduler	Payments (exactly-once)	leases and idempotent job execution
Video streaming	Object storage	chunked storage, CDN, adaptive bitrate
Nearby friends	Ride matching	both parties move; privacy controls who is visible
Stock exchange order book	Payments and booking	strict ordering per instrument on a single writer

The follow-ups that close the round

Prepare a crisp answer for each of these against every case you practise.

- 1 **"What happens at 10x load?"** Name the first component to saturate, the metric that would show it, and the change you would make.
- 2 **"What if a data centre is lost?"** State what is replicated across regions, what the recovery point and recovery time are, and which features degrade.
- 3 **"How do you deploy a schema change?"** Expand-and-contract: add the new field, dual-write, backfill, switch reads, then remove the old field.
- 4 **"What do you monitor?"** The SLIs for each user-facing flow, plus the saturation signal of the component you expect to fail first.
- 5 **"Where is the single point of failure?"** There usually is one - a coordinator, a registry, a primary database. Name it and its failover.
- 6 **"How would you test this?"** Load tests at the estimated peak, fault injection for the failure cases you described, and reconciliation for anything involving money or inventory.

Scoring a mock round

Score each mock from 0 to 2 on six dimensions:

TEXT				
	0	1	2	
requirements	none written	functional only	functional, non-functional, non-goals	
estimation	no numbers	numbers, not used	numbers that chose the design	
high-level design	unlabelled boxes	flows drawn	every arrow and store labelled	
deep dive	easy component	right component, shallow	right component, alternatives, failure	modes
trade-offs	none stated	stated when asked	volunteered, with a decision	
failure and scale	not discussed	mentioned	failure behaviour and first bottleneck	named

Nine or more out of twelve is a strong SDE-2 round. Keep the sheets: the dimension that stays lowest across attempts is what to practise next, and for most candidates it is estimation or failure behaviour rather than architecture.

Case bank

Practise each at 45 minutes, and score it.

Unique allocation: URL shortener with custom aliases; concert ticketing with seat maps; coupon code issuance with per-user limits.

Partitioning and skew: distributed cache with hot-key handling; leaderboard for 100 million players; key-value store with tunable consistency.

Ordering and delivery: one-to-one and group chat; collaborative document editing; payment system with refunds.

Precomputation and freshness: typeahead; home feed with ranking; trending topics over the last hour.

Spatial and moving data: ride matching; restaurant search by distance; delivery-driver dispatch.

Volume and durability: metrics and alerting; object storage; log aggregation and search.

WATCH OUT | Failure modes and common mistakes

- **Treating every prompt as new.** Most are a known difficulty in a new costume.
- **Unprepared follow-ups.** The last ten minutes are predictable; prepare them.
- **Practising without scoring.** Repetition without a score strengthens what is already strong.

TRY IT | Interview questions and model answers

Q: You have never seen this prompt. How do you start? A: The same way as any other: requirements, non-goals and an estimate. Then I identify the dominant difficulty - allocation, skew, ordering, precomputation, spatial data, or volume - and reuse the reasoning I know for it.

Q: What happens if a region goes down? A: I say what is replicated across regions and how - synchronously for money and inventory, asynchronously for most other data - which gives the recovery point, and how traffic fails over, which gives the recovery time. Then I name the features that degrade rather than fail.

Q: How do you change a schema without downtime? A: Expand and contract: add the new column, write to both, backfill, switch reads to the new column, then stop writing and drop the old one. Each step is reversible until the last.

TRY IT | Exercises

- 1 Pick three prompts from the mapping table that you have not practised. Identify the dominant difficulty of each in two minutes, then run a full 45-minute mock on one.
- 2 Write your prepared answer to each of the six follow-ups for the case you know best.
- 3 Score your last three mock rounds with the rubric. Which dimension is lowest?

RECAP | Chapter summary

New prompts are usually one of six known difficulties: unique allocation, skew, ordering, precomputation, spatial data, or volume. Recognise the difficulty early and reuse the matching reasoning. Prepare the six standard follow-ups, and score every mock on six dimensions so that practice targets your weakest one.

TRY IT | Revision checklist

- ☐ I can name the dominant difficulty of each case in this book.
- ☐ I can map five unfamiliar prompts onto those difficulties.
- ☐ I have prepared answers to the six standard follow-ups.
- ☐ I score every mock round out of twelve.

SDE-2 CORE CHAPTER

Chapter 11 - High-Level Design Case Drills

Work these in order. Each drill names the artifact to produce. Write your answer before reading the solution; the gap between the two is the lesson. Show arithmetic, and round in the direction that makes the design safer.

Estimation and allocation

D1. Scale the shortener tenfold. The URL shortener from chapter 2 now receives 1 billion new links per month, still 100 reads per write, 500 bytes per record, 5 years of retention. *Produce:* average writes and reads per second, records and storage over five years, and the one design decision that changes because of the new numbers.

D2. Size a random key. At 100 million new links per month for five years, random base62 keys must have fewer than one expected collision in total. *Produce:* the minimum key length, with the birthday arithmetic for the lengths either side of it.

Partitioning and skew

D3. Resize a cluster. A cache cluster grows from 10 to 12 nodes. *Produce:* the fraction of keys that change owner under `hash mod N`, the ideal fraction under consistent hashing, and one sentence on what the difference means for the database behind the cache.

D4. Choose quorums. A replicated store has $N = 3$. *Produce:* R and W for a shopping cart, a session token, and an account's email address, each with the consistency behaviour it gives.

D5. Move the feed threshold. In the Zipf follower model from chapter 4 (1,000,000 accounts, the rank- r account has top / r followers), the top account grows from 10 million to 100 million followers. *Produce:* the number of accounts above 10,000 and above 100,000 followers, and the share of fan-out writes each threshold removes, for both values of `top`. Say whether you would move the threshold.

Ordering and delivery

D6. Size the gateways. 50 million concurrent chat users, 200,000 connections per gateway, three availability zones. *Produce:* the number of gateways needed to serve the load, and the number needed to survive the loss of one zone.

D7. Trace the receiver. A conversation receiver has delivered up to sequence 7. Arrivals: 9, 8, 8, 11, 10, 12. *Produce:* what is delivered after each arrival, which gaps are requested, and which arrivals are dropped as duplicates.

Precomputation, spatial data, and volume

D8. Size a crawler's Bloom filter. A crawler must remember 1 billion URLs with a 0.1% false-positive rate.

Produce: bits, memory, and number of hash functions, plus what a false positive costs the crawler.

D9. Pick a geohash precision. "Restaurants within 200 metres" in a city near the equator. *Produce:* the precision, the cell size, the number of cells a query reads, and why.

D10. Size a larger metrics system. 20 million series at 15-second resolution, with the tiering from chapter 9 (15 s for 14 days, 1 minute for 90 days, 1 hour for a year) and 1.37 bytes per compressed point. *Produce:* points per day, raw and compressed GB per day, compressed storage for a year at full resolution, and tiered storage.

D11. Compare durability schemes. Per-node failure probability within one repair window: 1%. *Produce:* the object loss probability and overhead for 2x replication, 3x replication, RS(10,4), and RS(12,4), and the scheme you would choose for a backup tier.

Challenge

D12. Full mock: design a ticketing platform for stadium concerts. 50,000 seats per show, 2 million users arriving when a popular sale opens, seat maps, holds for 10 minutes, and payment. *Produce:* requirements and non-goals, the estimate, the high-level design, one deep dive of your choice, the follow-up answers for 10x load and region loss, and your rubric score out of twelve.

SDE-2 CORE CHAPTER

Chapter 12 - High-Level Design Case Drills: Worked Solutions

Every number below was computed by a program. Where a solution rests on an assumption the drill does not state, it says so; in an interview, a named assumption is worth more than a confident guess.

Estimation and allocation

D1. Scale the shortener tenfold

TEXT

writes/s	1,000,000,000 / 2,592,000 s	=	385.8 average
reads/s	x 100	=	38,580 average
records	1e9 x 12 x 5	=	60,000,000,000
storage	x 500 bytes	=	30 TB

What changes: **storage now needs partitioning**. Thirty terabytes plus replicas is beyond a comfortable single node, so the links table is partitioned by short code - with consistent hashing or a slot table, as in chapter 3. Read traffic of about 39,000 per second on average, and several times that at peak, also makes the cache mandatory rather than an optimisation.

What does not change: counter-range key generation. At 386 writes per second, blocks of 10,000 ids still mean a coordination request every 26 seconds across the whole fleet.

D2. Size a random key

Expected collisions for $n = 6,000,000,000$ keys in a space of 62^L :

TEXT

L = 10:	$n^2 / (2 \times 62^{10}) = 21.4$	expected collisions
L = 11:	$n^2 / (2 \times 62^{11}) = 0.35$	expected collisions
L = 12:	$n^2 / (2 \times 62^{12}) = 0.0056$	

Eleven characters is the minimum. Compare with the seven characters a counter needs for the same volume: random keys cost four extra characters to avoid a collision check that a counter never needs. A collision check with retry at seven characters is also valid - but say that it happens about five million times.

Partitioning and skew

D3. Resize a cluster

TEXT

```
hash mod N, 10 -> 12: 83.3% of keys move
consistent hashing:    2/12 = 16.7% move (the share taken over by the two new nodes)
```

For $\text{mod } N$, a key stays only if $h \bmod 10 == h \bmod 12$, which holds for 10 of every 60 residues. A cache that loses five-sixths of its placements at once sends almost the full read load to the database at the moment capacity was being added. With a ring, one-sixth of keys miss once and warm up again.

D4. Choose quorums

- **Shopping cart:** $W = 1, R = 1$ (or $W = 1$ with merge on read). Availability matters more than strict consistency; concurrent updates are merged, not overwritten, which needs version vectors.
- **Session token:** $W = 2, R = 1$ or $W = 1, R = 1$ with a short TTL. A token that is briefly missing on one replica causes a re-login, not data loss; choose by how annoying that is.
- **Account email:** $W = 2, R = 2$. $R + W > N$, so a read after a change sees the change. Showing an old email after the user updated it is a visible bug.

D5. Move the feed threshold

TEXT

```
top = 10,000,000 followers (143,435,683 writes/day if all pushed)
> 10,000 followers:      999 accounts, remove 52% of fan-out writes
> 100,000 followers:     99 accounts, remove 36%

top = 100,000,000 followers (1,438,773,858 writes/day if all pushed)
> 10,000 followers:     9,999 accounts, remove 68%
> 100,000 followers:    999 accounts, remove 52%
```

With the heavier tail, a 100,000-follower threshold now removes the same 52% that 10,000 did before, while pulling for ten times fewer accounts than a 10,000 threshold would. **Move the threshold up** if the read-side cost of merging many pulled accounts matters; keep it at 10,000 if write load is the constraint. The point of the drill is that the threshold is a number chosen from the distribution, not a constant.

Ordering and delivery

D6. Size the gateways

TEXT

```
load:                50,000,000 / 200,000 = 250 gateways
survive loss of 1 of 3 AZ: 250 x 3 / 2      = 375 gateways (125 per zone)
```

With 125 per zone, losing one zone leaves 250, exactly the load. Add a margin on top for reconnect storms: when a zone fails, its users all reconnect at once.

D7. Trace the receiver

Starting state: delivered through 7, so the next expected sequence is 8.

TEXT

```
arrive 9    -> buffer 9; gap requested for 8; nothing delivered
arrive 8    -> deliver 8, 9
arrive 8    -> duplicate (8 < next expected 10); dropped
arrive 11   -> buffer 11; gap requested for 10
arrive 10   -> deliver 10, 11
arrive 12   -> deliver 12
delivered in order: 8, 9, 10, 11, 12; gaps requested: 8 and 10; one duplicate dropped
```

Precomputation, spatial data, and volume

D8. Size a crawler's Bloom filter

TEXT

```
m = -n ln(p) / (ln 2)^2 = 14,377,587,566 bits = 1.80 GB (1,714 MiB)
k = (m / n) ln 2        = 9.97 -> 10 hash functions
```

About 14.4 bits per URL, however long the URLs are. A false positive means the crawler believes it has seen a URL it has not, so it **skips a page it should have crawled** - one in a thousand new URLs is missed. That is acceptable for a general crawler and not for one that must be complete, which would confirm positives against a durable store.

D9. Pick a geohash precision

Precision 6, whose cells are about 1.2 km by 611 m at the equator. The query reads the rider's cell and its **eight neighbours** and filters by true distance.

The tempting answer is precision 7, because its 153-metre cells are closer to the 200-metre radius. It is wrong with a 3 x 3 search: a rider standing at the edge of the centre cell can have a restaurant 200 m away that lies beyond the 153-metre neighbouring cell. The rule is to choose the finest precision whose cells are at least the radius on each side. If precision 6 returns too many candidates to filter, precision 7 with a 5 x 5 block (25 cells, 765 m across) also covers the radius.

D10. Size a larger metrics system

TEXT

```
points/day      20,000,000 x 86,400 / 15 = 115,200,000,000
raw              x 16 bytes              = 1,843 GB/day
compressed      x 1.37 bytes             = 158 GB/day
one year at 15 s, compressed             = 57.6 TB
tiered (15 s 14 d, 1 m 90 d, 1 h 1 y)    = 6.0 TB
```

Most of the tiered total comes from the 90 days of one-minute rollups. If six terabytes is too much, shortening that tier is the next lever.

D11. Compare durability schemes

TEXT			
2x replication	overhead 2.0x	P(loss)	1.0e-04
3x replication	overhead 3.0x	P(loss)	1.0e-06
RS(10,4)	overhead 1.4x	P(loss)	1.9e-07
RS(12,4)	overhead 1.33x	P(loss)	4.0e-07

For a backup tier - large, cold, written once, rarely read - **RS(10,4)**: the best durability here at 1.4x overhead. RS(12,4) saves a little storage for about twice the loss probability. 2x replication is a hundred times worse than 3x, which is why it is rarely used for data that must not be lost. These figures assume independent failures, so fragments must be spread across failure domains to come anywhere near them.

Challenge

D12. Full mock: stadium ticketing

Requirements. Browse a show's seat map; hold up to 6 seats for 10 minutes; pay; receive tickets.

Non-functional: no seat sold twice; the sale must stay up under the opening surge; the seat map may be a few seconds stale. Non-goals: resale, dynamic pricing, and venue management.

Estimate. 2 million users arriving over roughly the first minute is about 33,000 arrivals per second - but only 50,000 seats exist, so at most about 8,300 successful holds if everyone takes six. The inventory is tiny; the arrival rate is huge. **The problem is admission control, not storage.**

High-level design.

TEXT	
users -> CDN (static seat map, cached for seconds) -> waiting room -> booking service -> inventory DB	
	payment service (chapter 8)

- **Waiting room:** a queue that admits users at a rate the booking service can serve - say a few thousand per second - in arrival order, with a position shown to the user. Admission is a signed, short-lived token.
- **Booking service:** holds seats with a conditional update from **AVAILABLE** to **HELD**, all-or-nothing for multi-seat holds within one transaction, with a 10-minute expiry.
- **Payment:** idempotent, keyed by the hold id; on success the hold becomes a booking.
- **Seat map:** served from a cache refreshed every few seconds. A user can see a seat as available that was just held; the hold attempt fails and the map refreshes.

Deep dive: contention on the inventory database. 50,000 seat rows fit on one database. Contention is on individual rows, and conditional updates resolve it correctly - losers retry with other seats. The waiting room keeps the attempt rate bounded, so the database never sees the full surge.

Follow-ups. At 10x - 20 million users - only the waiting room grows, because admission caps the rate reaching the booking service; the waiting room itself is a stateless tier over a partitioned queue. Region loss: inventory is replicated synchronously to a second region, because an unreplicated hold or sale is not acceptable; the waiting room restarts from its persisted queue positions, and admitted tokens remain valid.

Rubric. Score honestly out of twelve against chapter 10. Common deductions: no admission control (the database takes the whole surge), per-seat holds that are not all-or-nothing, and payment not keyed by the hold.

SDE-2 CORE CHAPTER

Chapter 13 - Executable High-Level Design Model

This dependency-free Java 21 model recomputes and checks every number the cases quote: shortener estimates and birthday collisions, consistent-hashing balance and remapping, power-law fan-out, chat ordering and presence load, Bloom filter sizing and measured false positives, geohash cells and the boundary problem, idempotent payments with a double-entry ledger, seat contention, and storage durability and volume.

JAVA (continued 1/19)

```
import java.util.ArrayList;
import java.util.BitSet;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.NavigableMap;
import java.util.Set;
import java.util.SplittableRandom;
import java.util.TreeMap;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReferenceArray;

/**
 * Executable model of the arithmetic and algorithms in the HLD case book.
 *
 * Every number quoted in the chapters is printed by this program, and every
 * claim that can be checked is checked. Randomized experiments use fixed
 * seeds, so the output is identical on every run. Run it with:
 *
 *     javac --release 21 HighLevelDesignModel.java && java -ea HighLevelDesignModel
 *
 * Sections map to chapters:
 * 1. Estimation and key space for a URL shortener           (ch 2)
 * 2. Consistent hashing: balance and remapping             (ch 3)
 * 3. Feed fan-out on a power-law follower graph            (ch 4)
 * 4. Chat ordering, gap detection, and presence load        (ch 5)
 * 5. Bloom filters: sizing and measured false positives     (ch 6)
 * 6. Geohash cells and the boundary problem                 (ch 7)
 * 7. Idempotent payments, double-entry ledger, seat contention (ch 8)
 * 8. Storage durability and time-series volume              (ch 9)
 */
public final class HighLevelDesignModel {
    private HighLevelDesignModel() {}
}
```

JAVA (continued 2/19)

```
static void check(boolean condition, String message) {
    if (!condition) {
        throw new IllegalStateException("check failed: " + message);
    }
}

static final long SECONDS_PER_DAY = 86_400;
static final long SECONDS_PER_MONTH = 30 * SECONDS_PER_DAY;

/** SplitMix64 finalizer: a fast, well-mixed 64-bit hash of a 64-bit input. */
static long mix64(long z) {
    z = (z ^ (z >>> 30)) * 0xbf58476d1ce4e5b9L;
    z = (z ^ (z >>> 27)) * 0x94d049bb133111ebL;
    return z ^ (z >>> 31);
}

static long hash(String key) {
    long h = 1125899906842597L;
    for (int i = 0; i < key.length(); i++) {
        h = 31 * h + key.charAt(i);
    }
    return mix64(h);
}

// -----
// 1. URL shortener: estimation and key space
// -----

static final String BASE62 = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

static String toBase62(long value) {
    if (value == 0) {
        return "0";
    }
    StringBuilder out = new StringBuilder();
```

JAVA (continued 3/19)

```
    long v = value;
    while (v > 0) {
        out.append(BASE62.charAt((int) (v % 62)));
        v /= 62;
    }
    return out.reverse().toString();
}

static long fromBase62(String text) {
    long v = 0;
    for (int i = 0; i < text.length(); i++) {
        v = v * 62 + BASE62.indexOf(text.charAt(i));
    }
    return v;
}

static void urlShortener() {
    System.out.println("== 1. URL shortener ==");
    long newPerMonth = 100_000_000L;
    int readsPerWrite = 100;
    int bytesPerRecord = 500;
    int years = 5;

    double writesPerSecond = (double) newPerMonth / SECONDS_PER_MONTH;
    double readsPerSecond = writesPerSecond * readsPerWrite;
    long totalRecords = newPerMonth * 12 * years;
    double storageTb = (double) totalRecords * bytesPerRecord / 1e12;
    System.out.printf(
        "    writes/s %.1f average, reads/s %,.0f average%n",
        writesPerSecond, readsPerSecond);
    System.out.printf(
        "    %d years -> %,d records, %.1f TB at %d bytes each%n",
        years, totalRecords, storageTb, bytesPerRecord);
    check(Math.abs(writesPerSecond - 38.58) < 0.01, "write rate");
    check(totalRecords == 6_000_000_000L, "record count");
}
```

JAVA (continued 4/19)

```
double keySpace7 = Math.pow(62, 7);
double counterYears = keySpace7 / writesPerSecond / (365.0 * SECONDS_PER_DAY);
System.out.printf(
    " 62^7 = %.3e keys; a counter lasts %,.0f years at this rate%n",
    keySpace7, counterYears);

double expectedCollisions = (double) totalRecords * totalRecords / (2 * keySpace7);
System.out.printf(
    "  random 7-char keys for %d URLs: ~%,.0f expected collisions%n",
    totalRecords, expectedCollisions);
check(expectedCollisions > 1e6, "truncated random keys must handle collisions");

int smallSpace = 62 * 62 * 62;
int draws = 2_000;
int trials = 400;
double predicted = (double) draws * (draws - 1) / (2.0 * smallSpace);
SplittableRandom random = new SplittableRandom(17);
long observed = 0;
for (int trial = 0; trial < trials; trial++) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < draws; i++) {
        int key = random.nextInt(smallSpace);
        int before = seen.merge(key, 1, Integer::sum) - 1;
        observed += before;
    }
}
double observedMean = (double) observed / trials;
System.out.printf(
    "  birthday check, %d keys into 62^3: predicted %.2f colliding pairs,"
    + " measured %.2f over %d trials%n",
    draws, predicted, observedMean, trials);
check(Math.abs(observedMean - predicted) / predicted < 0.05, "birthday estimate holds");

long id = 125_000_000_000L;
String code = toBase62(id);
System.out.printf(
```

JAVA (continued 5/19)

```

        " counter id %,d -> base62 \"%s\" (%d chars)%n", id, code, code.length());
        check(fromBase62(code) == id && code.length() == 7, "base62 round trip");
    }

    // -----
    // 2. Consistent hashing
    // -----

    static final class HashRing {
        private final NavigableMap<Long, String> ring = new TreeMap<>();
        private final int virtualNodes;

        HashRing(int virtualNodes) {
            this.virtualNodes = virtualNodes;
        }

        void add(String node) {
            for (int v = 0; v < virtualNodes; v++) {
                ring.put(hash(node + "#" + v), node);
            }
        }

        String owner(String key) {
            Map.Entry<Long, String> e = ring.ceilingEntry(hash(key));
            return e != null ? e.getValue() : ring.firstEntry().getValue();
        }
    }

    static double maxOverMean(Map<String, Integer> load) {
        double mean = load.values().stream().mapToInt(Integer::intValue).average().orElse(0);
        int max = load.values().stream().mapToInt(Integer::intValue).max().orElse(0);
        return max / mean;
    }

    static void consistentHashing() {
        System.out.println();
    }

```

JAVA (continued 6/19)

```

System.out.println("== 2. Consistent hashing: 10 nodes, 1,000,000 keys ==");
int nodes = 10;
int keys = 1_000_000;
double[] expectedBands = {Double.MAX_VALUE, Double.MAX_VALUE, 1.30, 1.15};
int[] vnodeCounts = {1, 10, 100, 200};
for (int i = 0; i < vnodeCounts.length; i++) {
    HashRing ring = new HashRing(vnodeCounts[i]);
    for (int n = 0; n < nodes; n++) {
        ring.add("node-" + n);
    }
    Map<String, Integer> load = new HashMap<>();
    for (int k = 0; k < keys; k++) {
        load.merge(ring.owner("key-" + k), 1, Integer::sum);
    }
    double ratio = maxOverMean(load);
    System.out.printf(
        "    %3d virtual nodes each: busiest node holds %.2fx the mean%n",
        vnodeCounts[i], ratio);
    check(ratio < expectedBands[i], "balance improves with virtual nodes");
}

HashRing before = new HashRing(100);
HashRing after = new HashRing(100);
for (int n = 0; n < nodes; n++) {
    before.add("node-" + n);
    after.add("node-" + n);
}
after.add("node-" + nodes);
int ringMoved = 0;
int moduloMoved = 0;
for (int k = 0; k < keys; k++) {
    String key = "key-" + k;
    if (!before.owner(key).equals(after.owner(key))) {
        ringMoved++;
    }
}
long h = hash(key) & Long.MAX_VALUE;

```

JAVA (continued 7/19)

```

        if (h % nodes != h % (nodes + 1)) {
            moduloMoved++;
        }
    }
    double ringFraction = (double) ringMoved / keys;
    double moduloFraction = (double) moduloMoved / keys;
    System.out.printf(
        "    add an 11th node: ring moves %.1f%% of keys, hash mod N moves %.1f%%\n",
        ringFraction * 100, moduloFraction * 100);
    System.out.printf("    ideal is 1/11 = %.1f%%\n", 100.0 / (nodes + 1));
    check(Math.abs(ringFraction - 1.0 / (nodes + 1)) < 0.02, "ring moves about 1/(N+1)");
    check(moduloFraction > 0.85, "modulo moves almost everything");
}

// -----
// 3. Feed fan-out
// -----

static void feedFanOut() {
    System.out.println();
    System.out.println("== 3. Feed fan-out: 1,000,000 accounts, Zipf followers ==");
    int accounts = 1_000_000;
    long topFollowers = 10_000_000L;
    long[] followers = new long[accounts];
    long total = 0;
    for (int rank = 1; rank <= accounts; rank++) {
        followers[rank - 1] = topFollowers / rank;
        total += followers[rank - 1];
    }
    System.out.printf(
        "    one post per account per day, push model: %d timeline writes/day\n", total);
    System.out.printf(
        "    that is %,.0f writes/s on average\n", (double) total / SECONDS_PER_DAY);

    for (long threshold : new long[] {100_000L, 10_000L, 1_000L}) {
        long pushed = 0;

```

JAVA (continued 8/19)

```

        int celebrities = 0;
        for (long f : followers) {
            if (f > threshold) {
                celebrities++;
            } else {
                pushed += f;
            }
        }
        double saved = 1.0 - (double) pushed / total;
        System.out.printf(
            "    pull for accounts over %,7d followers: %,6d accounts (%.3f%%)"
            + "    remove %.0f%% of fan-out writes%n",
            threshold, celebrities, 100.0 * celebrities / accounts, saved * 100);
        if (threshold == 10_000L) {
            check(celebrities == 999, "999 accounts exceed 10,000 followers");
            check(saved > 0.5, "0.1% of accounts cause over half the writes");
        }
    }
    long worst = followers[0];
    double seconds = worst / 50_000.0;
    System.out.printf(
        "    top account's single post at 50,000 writes/s: %,0f s to fan out%n", seconds);
}

// -----
// 4. Chat ordering and presence
// -----

/** Per-conversation receiver: delivers in sequence order, buffers early arrivals. */
static final class ConversationReceiver {
    private long nextExpected = 1;
    private final TreeMap<Long, String> buffered = new TreeMap<>();
    private final List<String> delivered = new ArrayList<>();
    private final Set<Long> gapsRequested = new HashSet<>();

    void receive(long sequence, String body) {

```

JAVA (continued 9/19)

```

        if (sequence < nextExpected || buffered.containsKey(sequence)) {
            return;
        }
        buffered.put(sequence, body);
        while (buffered.containsKey(nextExpected)) {
            delivered.add(buffered.remove(nextExpected));
            nextExpected++;
        }
        if (!buffered.isEmpty()) {
            for (long s = nextExpected; s < buffered.firstKey(); s++) {
                gapsRequested.add(s);
            }
        }
    }
}

static void chat() {
    System.out.println();
    System.out.println("== 4. Chat: ordering and presence ==");
    ConversationReceiver receiver = new ConversationReceiver();
    long[] arrivals = {1, 2, 4, 5, 2, 3, 6};
    for (long s : arrivals) {
        receiver.receive(s, "m" + s);
    }
    System.out.printf(
        " arrivals %s -> delivered %s, gap requested for %s\n",
        java.util.Arrays.toString(arrivals), receiver.delivered, receiver.gapsRequested);
    check(
        receiver.delivered.equals(List.of("m1", "m2", "m3", "m4", "m5", "m6")),
        "in-order, exactly-once delivery despite a gap and a duplicate");
    check(receiver.gapsRequested.equals(Set.of(3L)), "the missing sequence is requested");

    long online = 10_000_000L;
    int heartbeatSeconds = 30;
    int missedBeforeOffline = 3;
    System.out.printf(

```

JAVA (continued 10/19)

```

        " %d online users, heartbeat every %ds: %d heartbeats/s\n",
        online, heartbeatSeconds, online / heartbeatSeconds);
    System.out.printf(
        " offline after %d missed beats: detected %d-%d s after disconnect\n",
        missedBeforeOffline,
        (missedBeforeOffline - 1) * heartbeatSeconds,
        missedBeforeOffline * heartbeatSeconds);
    long connectionsPerServer = 500_000L;
    System.out.printf(
        " at %d connections per gateway: %d gateways before headroom\n",
        connectionsPerServer, (online + connectionsPerServer - 1) / connectionsPerServer);
}

// -----
// 5. Bloom filter
// -----

static final class BloomFilter {
    private final BitSet bits;
    private final int m;
    private final int k;

    BloomFilter(int m, int k) {
        this.bits = new BitSet(m);
        this.m = m;
        this.k = k;
    }

    private int index(long h1, long h2, int i) {
        return (int) Long remainderUnsigned(h1 + i * h2, m);
    }

    void add(String key) {
        long h1 = hash(key);
        long h2 = mix64(h1) | 1;
        for (int i = 0; i < k; i++) {

```

JAVA (continued 11/19)

```

        bits.set(index(h1, h2, i));
    }
}

boolean mightContain(String key) {
    long h1 = hash(key);
    long h2 = mix64(h1) | 1;
    for (int i = 0; i < k; i++) {
        if (!bits.get(index(h1, h2, i))) {
            return false;
        }
    }
    return true;
}
}

static void bloom() {
    System.out.println();
    System.out.println("== 5. Bloom filter: 100,000 members, 1% target ==");
    int n = 100_000;
    double target = 0.01;
    int m = (int) Math.ceil(-n * Math.log(target) / (Math.log(2) * Math.log(2)));
    int k = (int) Math.round((double) m / n * Math.log(2));
    double theoretical = Math.pow(1 - Math.exp(-(double) k * n / m), k);
    System.out.printf(
        "    m = %d bits (%.0f KB), k = %d hashes, predicted FP %.3f%%\n",
        m, m / 8.0 / 1024, k, theoretical * 100);
    BloomFilter filter = new BloomFilter(m, k);
    for (int i = 0; i < n; i++) {
        filter.add("user-" + i);
    }
    for (int i = 0; i < n; i++) {
        check(filter.mightContain("user-" + i), "no false negatives");
    }
    int falsePositives = 0;
    int probes = 200_000;
}

```

JAVA (continued 12/19)

```
    for (int i = 0; i < probes; i++) {
        if (filter.mightContain("other-" + i)) {
            falsePositives++;
        }
    }
    double measured = (double) falsePositives / probes;
    System.out.printf(
        "    measured on %,d non-members: %.3f%%; 0 false negatives on %,d members%n",
        probes, measured * 100, n);
    check(Math.abs(measured - theoretical) < 0.002, "measurement matches the formula");
}

// -----
// 6. Geohash
// -----

static final String GE032 = "0123456789bcdefghjkmnpqrstuvwxyz";

static String geohash(double lat, double lon, int precision) {
    double latLo = -90;
    double latHi = 90;
    double lonLo = -180;
    double lonHi = 180;
    StringBuilder out = new StringBuilder();
    boolean even = true;
    int bit = 0;
    int ch = 0;
    while (out.length() < precision) {
        if (even) {
            double mid = (lonLo + lonHi) / 2;
            if (lon >= mid) {
                ch = (ch << 1) | 1;
                lonLo = mid;
            } else {
                ch <<= 1;
                lonHi = mid;
            }
        }
    }
}
```

JAVA (continued 13/19)

```

        }
    } else {
        double mid = (latLo + latHi) / 2;
        if (lat >= mid) {
            ch = (ch << 1) | 1;
            latLo = mid;
        } else {
            ch <<= 1;
            latHi = mid;
        }
    }
    even = !even;
    if (++bit == 5) {
        out.append(GE032.charAt(ch));
        bit = 0;
        ch = 0;
    }
}
return out.toString();
}

/** Longitude bounds of the cell containing (lat, lon) at the given precision. */
static double[] lonBounds(double lon, int precision) {
    int lonBits = (5 * precision + 1) / 2;
    double width = 360.0 / (1L << lonBits);
    double lo = Math.floor((lon + 180) / width) * width - 180;
    return new double[] {lo, lo + width};
}

static void geohashes() {
    System.out.println();
    System.out.println("== 6. Geohash ==");
    double kmPerDegree = 111.32;
    System.out.println("  precision    cell at the equator (width x height)");
    for (int p = 4; p <= 8; p++) {
        int bits = 5 * p;

```

JAVA (continued 14/19)

```

        int lonBits = (bits + 1) / 2;
        int latBits = bits / 2;
        double widthKm = 360.0 / (1L << lonBits) * kmPerDegree;
        double heightKm = 180.0 / (1L << latBits) * kmPerDegree;
        System.out.printf(" %d          %s x %s\n", p, distance(widthKm), distance(heightKm));
    }

    double lat = 37.7749;
    double lon = -122.4194;
    String sf = geohash(lat, lon, 8);
    System.out.printf(" San Francisco (37.7749, -122.4194) -> %s\n", sf);
    check(sf.startsWith("9q8yy"), "known geohash prefix for San Francisco");

    double edge = lonBounds(lon, 6)[1];
    double west = edge - 0.00003;
    double east = edge + 0.00003;
    double metres = (east - west) * kmPerDegree * 1000 * Math.cos(Math.toRadians(lat));
    String a = geohash(lat, west, 6);
    String b = geohash(lat, east, 6);
    int common = 0;
    while (common < 6 && a.charAt(common) == b.charAt(common)) {
        common++;
    }
    System.out.printf(
        " two points %.1f m apart across a cell edge: %s vs %s (%d shared chars)\n",
        metres, a, b, common);
    check(!a.equals(b), "nearby points can land in different cells");
}

static String distance(double km) {
    return km >= 1 ? String.format("%.1f km", km) : String.format("%.0f m", km * 1000);
}

// -----
// 7. Payments and booking
// -----

```

JAVA (continued 15/19)

```
record Entry(String account, long cents) {}

static final class PaymentService {
    private final Map<String, String> idempotency = new ConcurrentHashMap<>();
    private final List<List<Entry>> journal = new ArrayList<>();
    private final AtomicInteger charges = new AtomicInteger();

    synchronized String charge(String idempotencyKey, String customer, long cents) {
        String previous = idempotency.get(idempotencyKey);
        if (previous != null) {
            return previous;
        }
        String paymentId = "pay-" + (journal.size() + 1);
        journal.add(
            List.of(
                new Entry("customer:" + customer, -cents),
                new Entry("merchant:revenue", cents)));
        charges.incrementAndGet();
        idempotency.put(idempotencyKey, paymentId);
        return paymentId;
    }

    long balance(String account) {
        long total = 0;
        for (List<Entry> txn : journal) {
            for (Entry e : txn) {
                if (e.account().equals(account)) {
                    total += e.cents();
                }
            }
        }
        return total;
    }

    boolean everyTransactionBalances() {
```

JAVA (continued 16/19)

```

        for (List<Entry> txn : journal) {
            if (txn.stream().mapToLong(Entry::cents).sum() != 0) {
                return false;
            }
        }
        return true;
    }
}

static void payments() throws InterruptedException {
    System.out.println();
    System.out.println("== 7. Payments and booking ==");
    PaymentService service = new PaymentService();
    SplittableRandom random = new SplittableRandom(5);
    int orders = 100;
    int attempts = 0;
    for (int order = 0; order < orders; order++) {
        int deliveries = 1 + random.nextInt(4);
        for (int d = 0; d < deliveries; d++) {
            service.charge("order-" + order, "c" + (order % 7), 1_999);
            attempts++;
        }
    }
    System.out.printf(
        " %d orders, %d charge requests (retries included) -> %d charges%n",
        orders, attempts, service.charges.get());
    System.out.printf(
        " merchant revenue %s, every transaction sums to zero: %s%n",
        cents(service.balance("merchant:revenue")), service.everyTransactionBalances());
    check(service.charges.get() == orders, "one charge per order");
    check(service.balance("merchant:revenue") == orders * 1_999L, "revenue");
    check(service.everyTransactionBalances(), "double entry");

    int seats = 10;
    int buyers = 50;
    AtomicReferenceArray<String> seatHolder = new AtomicReferenceArray<>(seats);

```

JAVA (continued 17/19)

```

AtomicInteger booked = new AtomicInteger();
List<Thread> threads = new ArrayList<>();
for (int b = 0; b < buyers; b++) {
    String buyer = "buyer-" + b;
    int wanted = b % seats;
    Thread t =
        new Thread(
            () -> {
                if (seatHolder.compareAndSet(wanted, null, buyer)) {
                    booked.incrementAndGet();
                }
            }
        );
    threads.add(t);
    t.start();
}
for (Thread t : threads) {
    t.join();
}
Set<String> holders = new HashSet<>();
for (int s = 0; s < seats; s++) {
    holders.add(seatHolder.get(s));
}
System.out.printf(
    "    %d buyers race for %d seats with compare-and-set: %d booked,"
    + "    %d distinct holders%n",
    buyers, seats, booked.get(), holders.size());
check(booked.get() == seats && holders.size() == seats, "no seat sold twice");
}

static String cents(long c) {
    return String.format("$%,d.%02d", c / 100, c % 100);
}

// -----
// 8. Storage durability and time-series volume
// -----

```

JAVA (continued 18/19)

```

static double binomialTail(int n, int atLeast, double p) {
    double total = 0;
    for (int i = atLeast; i <= n; i++) {
        total += choose(n, i) * Math.pow(p, i) * Math.pow(1 - p, n - i);
    }
    return total;
}

static double choose(int n, int k) {
    double c = 1;
    for (int i = 1; i <= k; i++) {
        c = c * (n - k + i) / i;
    }
    return c;
}

static void storage() {
    System.out.println();
    System.out.println("== 8. Storage durability and time-series volume ==");
    double p = 0.01;
    System.out.printf("  chance a node fails before repair completes: %.0f%%\n", p * 100);
    double replication = binomialTail(3, 3, p);
    double rs63 = binomialTail(9, 4, p);
    double rs104 = binomialTail(14, 5, p);
    System.out.printf(
        "    3x replication      overhead 3.00x  P(loss per object) %.2e\n", replication);
    System.out.printf("    RS(6,3) erasure      overhead 1.50x  P(loss per object) %.2e\n", rs63);
    System.out.printf("    RS(10,4) erasure    overhead 1.40x  P(loss per object) %.2e\n", rs104);
    check(rs104 < replication, "RS(10,4) is more durable than 3x at under half the storage");

    long series = 1_000_000L;
    int intervalSeconds = 10;
    long pointsPerDay = series * (SECONDS_PER_DAY / intervalSeconds);
    double rawGb = pointsPerDay * 16 / 1e9;
    double compressedGb = pointsPerDay * 1.37 / 1e9;
}

```

JAVA (continued 19/19)

```

        System.out.printf(
            " %d series every %ds: %d points/day\n", series, intervalSeconds, pointsPerDay);
        System.out.printf(
            " 16 bytes/point raw: %.0f GB/day; ~1.37 bytes/point compressed:"
            + " %.1f GB/day\n",
            rawGb, compressedGb);
        double rawYear = rawGb * 365 / 1000;
        double compressedYear = compressedGb * 365 / 1000;
        double tiered =
            compressedGb * 14 / 1000
            + series * (1440L * 90) * 1.37 / 1e12
            + series * (24L * 365) * 1.37 / 1e12;
        System.out.printf(
            " one year at 10s: %.1f TB raw, %.2f TB compressed\n", rawYear, compressedYear);
        System.out.printf(
            " compressed and tiered (10s for 14d, 1m for 90d, 1h for 1y): %.2f TB\n", tiered);
        System.out.printf(
            " compression saves %.1fx; tiering saves a further %.1fx\n",
            rawYear / compressedYear, compressedYear / tiered);
        check(compressedYear / tiered > 10, "tiering matters as much as compression");
    }

    public static void main(String[] args) throws InterruptedException {
        urlShortener();
        consistentHashing();
        feedFanOut();
        chat();
        bloom();
        geohashes();
        payments();
        storage();
        System.out.println();
        System.out.println("All high-level design checks passed.");
    }
}

```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: the case method and the estimation toolkit
- Interview Core: shortener, cache, feed, and chat with their deep dives
- SDE-2 Follow-up: typeahead, proximity, payments, and storage durability
- Challenge: timed 45-minute mocks scored out of twelve

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: SD 04 - Low-Level Design and Machine Coding](#)

[Series index](#)

[Return to the complete series index](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design
System Design	SD 04	Low-Level Design and Machine Coding
System Design	SD 05	High-Level Design Case Book

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: System Design - Book 05 of 05 - Study Step SD-05. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.